

-Prova intermedia di Algoritmi-
(11 Aprile 2019)

Cognome Nome Matricola

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea sottostante il suo funzionamento e motivarne la correttezza.

Domanda 1 Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{1}{3} T(n-1) + 3n^2$$

ha soluzione in $\Theta(n^2)$.

Soluzione: Per provare che $T(n) = O(n^2)$ dobbiamo dimostrare che $T(n) \leq cn^2$, per un'opportuna costante $c > 0$. Procediamo per induzione:

$$\begin{aligned} T(n) &= \frac{1}{3} T(n-1) + 3n^2 \\ &\leq \frac{1}{3} c(n-1)^2 + 3n^2 \\ &\leq \frac{1}{3} cn^2 + 3n^2 \\ &\leq cn^2 \end{aligned}$$

dove, per la validità dell'ultima disuguaglianza $\frac{1}{3} cn^2 + 3n^2 \leq cn^2$, occorre che

$$\frac{2}{3} cn^2 \geq 3n^2$$

ovvero, $c \geq 9/2$, con n qualunque.

Per provare $T(n) = \Omega(n^2)$ dobbiamo dimostrare che $T(n) \geq dn^2$, per un'opportuna costante $d > 0$. La prova per induzione non usa neppure l'ipotesi induttiva. Infatti

$$T(n) = \frac{1}{3} T(n-1) + 3n^2 \geq dn^2$$

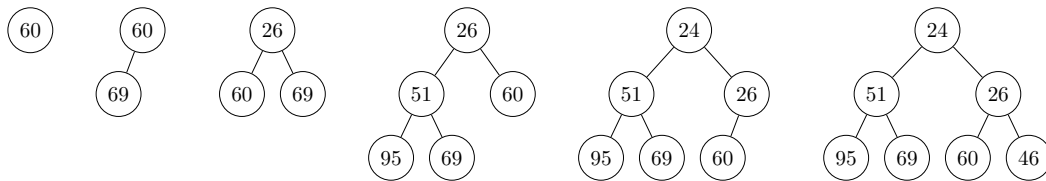
purché $d \leq 3$.

Domanda 2 Dare la definizione di min-heap. Data la sequenza di elementi 60, 69, 26, 95, 51, 24, 46, 80, 60, 38, 12, 70 si specifichi il min-heap ottenuto, inserendo uno alla volta questi elementi nell'ordine indicato, a partire da uno heap vuoto. Si descriva sinteticamente come si procede per arrivare al risultato.

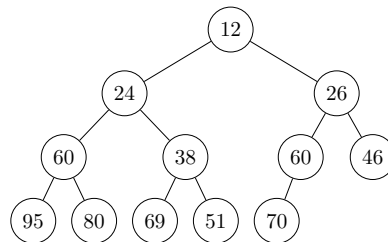
Soluzione: Un min-heap è un albero binario ordinato quasi-completo con la proprietà che per ogni nodo x , se x non è radice, il genitore di x ha chiave minore o uguale a quella di x , o equivalentemente ogni nodo x ha chiave minore o uguale a quella dei suoi successori.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MinHeapifyUp** per ripristinare la proprietà di min-heap.

I primi passi producono gli heap indicati in figura.



Il risultato finale è quindi:



che in forma di array, è [12, 24, 26, 60, 38, 60, 46, 95, 80, 69, 51, 70].

Domanda 3 Realizzare una funzione `Diff(A,k)` che, dato un array `A[1,n]` ordinato in senso crescente, verifica se esiste una coppia di indici `i, j` tali che $A[i] - A[j] = k$. Restituisce la coppia di indici se esiste e (0,0) altrimenti. La funzione non deve alterare l'input e deve operare in spazio costante. Scrivere lo pseudocodice e valutarne la complessità.

Soluzione:

Il codice può essere:

```
diff(A, n, k):
    i=1
    j=1
    while (i<=n) and (j<=n) and (A[i]- A[j] <> k)
        if (A[i]- A[j] < k)
            i++
        else
            j++

    if (i <= n) and (j<=n)
        return (i,j)
    else
        return (0,0)
```

È facile vedere che si mantiene l'invariante $\forall (i', j') \in [1, n]. (i' < i) \vee (j' < j) \Rightarrow A[i'] - A[j'] \neq k$, ovvero una coppia (i', j') tale che $A[i'] - A[j'] = k$ può esistere solo tra le coppie ancora esplorabili $(i' \geq i \text{ e } j' \geq j)$, ovvero, graficamente nella parte non grigia:

		<i>j</i>		
<i>i</i>		$A[i]-A[j]$		

Infatti, inizialmente, con $i = j = 1$, l'invariante è vacuamente vero.

Ad ogni iterazione, se entro nel ciclo, ci sono due possibilità:

- Se $A[i] - A[j] < k$, allora incremento i . In questo modo, escludo dall'esplorazione le coppie (i, j') con $j' \geq j$ per le quali, dato che l'array è crescente e quindi $A[j] \leq A[j']$, vale

$$A[i] - A[j'] \leq A[i] - A[j] < k.$$

Dunque non escludo coppie utili, l'invariante continua a valere.

- Dualmente, $A[i] - A[j] > k$, allora incremento j . In questo modo, escludo dall'esplorazione le coppie (i', j) con $i' \geq i$ per le quali, dato che l'array è crescente e quindi $A[i] \leq A[i']$, vale

$$A[i'] - A[j] \geq A[i] - A[j] > k.$$

Dunque, anche in questo caso, non escludo coppie utili, l'invariante continua a valere.

Quando esco dal ciclo, se $A[i] - A[j] = k$, ho concluso con successo. Altrimenti deve essere $i > n$ o $j > n$, che unitamente all'invariante, mi permettono di concludere che per ogni $i, j \in [1, n]$, $A[i] \neq A[j]$, come desiderato.

Da questo la correttezza segue immediatamente. La complessità è lineare. Il numero di iterazioni è pari al più a $2n - 1$, dato che i e j partono da 1, sono limitate da n ed ogni iterazione aumenta una delle due. Dato che ciascuna iterazione ha costo costante, ottengo $T(n) = O(2n - 1) = O(n)$.

Algoritmi e Strutture Dati

27 Giugno 2019

Cognome Nome Matricola

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea sottostante e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (5 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 4T(n/3) + n^2 + 1$$

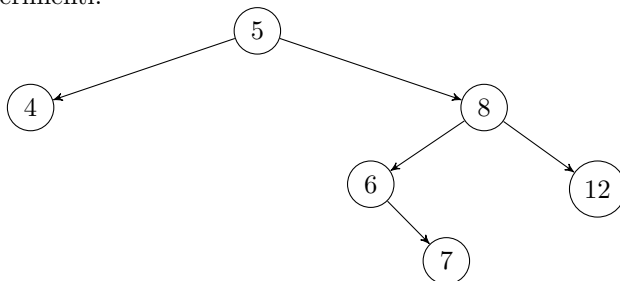
Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 3$, $f(n) = n^2 + 1$. Si osserva che $\log_b a = \log_3 4 < 2$ quindi $f(n) = \Omega(n^{\log_b a + \epsilon})$ (per $0 < \epsilon \leq 2 - \log_3 4$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{3}) = 4(\frac{n^2}{9} + 1) \leq cn^2 \leq c(n^2 + 1)$, asintoticamente, quando $4/9 < c < 1$. Più precisamente, per avere

$$4(\frac{n^2}{9} + 1) \leq cn^2$$

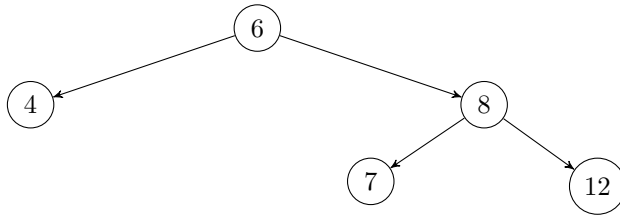
Occorre che $c \geq \frac{4}{9} + \frac{1}{n^2}$, che, per $n \geq 2$ si ottiene quando $c \geq \frac{4}{9} + \frac{1}{4} = \frac{25}{36} < 1$.

Domanda B (5 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi: 5, 4, 8, 6, 12, 7. Si supponga che dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione: Per la definizione di albero binario di ricerca e la descrizione delle procedure di inserimento e cancellazione si consulti il libro. Sullo specifico esempio si ottiene, dopo gli inserimenti:



La cancellazione di 5 quindi produce:



Domanda C (4 punti) Si consideri un insieme di 6 attività $a_i, 1 \leq i \leq 6$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (2, 3, 5, 2, 1, 9)$$

$$\mathbf{f} = (4, 7, 8, 9, 10, 11).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività: $\{a_1, a_3, a_6\}$

Esercizi

Esercizio 1 (8 punti) Realizzare una funzione booleana `triple(A)` che dato un array di numeri $A[1..n]$ verifica se esistono tre indici (non necessariamente diversi) i, j e k , tali che $A[i] + A[j] = A[k]$ e restituisce un corrispondente valore booleano. Ad esempio per $[85, 10, 29, 93, 95]$ restituisce true, dato che $85 + 10 = 95$, mentre per $[70, 45, 83, 21, 68]$ restituisce false. Si suggerisce di ordinare l'array. Valutare la complessità.

Soluzione: Assumiamo inizialmente che l'array A sia ordinato. In questo caso, si può cercare, per ogni indice k , una coppia (i, j) tale che $A[k] = A[i] + A[j]$.

assume $A[1..n]$ ordinato

cerca tre indici i, j, k tali che $A[i] + A[j] = A[k]$

```

Triple(A)
  n = A.length
  found = false
  k=1
  while (not found) and (k <= n)
    # cerca se ci sono due indici tali che  $A[i] + A[j] = A[k]$ 
    i=1
    j=n
    # invariante:
    #   per ogni  $i' < i$ , e  $j'$  qualunque  $A[i'] + A[j'] < A[k]$ 
    #   per ogni  $j' > j$ , e  $i'$  qualunque  $A[i'] + A[j'] > A[k]$ 
    while (i <= j) and ( $A[i] + A[j] \neq A[k]$ )
      if  $A[i] + A[j] < A[k]$ 
        i = i+1
      else
        j = j-1
    found = i <= j

```

k=k+1

return found

La complessità è $O(n^2)$: infatti, il ciclo interno, ad ogni iterazione incrementa i o decrementa j , quindi il numero massimo di iterazioni è n , e viene ripetuto al più n volte per effetto del ciclo esterno.

Se l'array non è ordinato, si può ordinarlo preventivamente, cosa che si può fare con costo $O(n \log n)$ e quindi senza alterare la complessità asintotica complessiva.

Esercizio 2 (8 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ (z(i-1, j) \cdot z(i, j+1)) - z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up $\text{COMPUTE_Z}(A)$ che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$.
2. Si valuti il numero esatto $T_{CZ}(n)$ di operazioni tra interi eseguite dall'algoritmo sviluppato al punto (a).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
COMPUTE_Z(A)
n = length(A)
for i=1 to n do
    z[1,i] = a_i
    z[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        z[i,j] = (z[i-1,j] * z[i,j+1]) - z[i-1,j+1]
return z[n,1]
```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonali parallele alla diagonale principale partendo da quella contenente solo $z[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due operazioni tra interi, e quindi

$$\begin{aligned} T_{CZ}(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due operazioni per ogni elemento di una tabella $(n - 1) \times (n - 1)$.

Nota: Correzione, risultati e visione dei compiti: *Giovedì 4 Luglio, ore 9:30 2BC/30*

Algoritmi e Strutture Dati
23 Agosto 2019

Cognome Nome Matricola

1. **Durata: 2:30 h**
2. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
3. Quando si presenta un algoritmo è fondamentale spiegare l'idea sottostante e motivarne la correttezza.
4. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
5. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione **bella copia o brutta copia.**

Domande

Domanda A (5 punti) Data la ricorrenza

$$T(n) = \begin{cases} 2 & \text{se } n = 0 \\ T(n-1) + 2n + 1 & \text{se } n > 0 \end{cases}$$

determinarne la soluzione esatta.

Domanda B (4 punti) Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m-2)$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 14, 22, 10, 16, 8.

Domanda C (5 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *oro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Esercizi

Esercizio 1 (7 punti)

- Scrivere una funzione $\text{Rep}(T, k)$ che dato un albero di ricerca T e una chiave k ritorna il numero di occorrenze di k in T . Valutare la complessità dell'algoritmo definito.
- Se si assume che l'albero binario di ricerca non contenga chiavi ripetute cosa cambia? (Ovviamente in questo caso il numero di occorrenze è al più uno). Adattare la soluzione e discutere la relativa complessità.

Esercizio 2 (9 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un $(2,1)$ -boxing che minimizza il numero di sottoinsiemi della partizione.

1. Scrivere il codice di un algoritmo greedy che restituisce un $(2,1)$ -boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

Nota: Correzione, risultati e visione dei compiti: 28 Agosto 2019, ore 14:30 (IBC/50)

Algoritmi e Strutture Dati
13 – 09 – 2019

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano i fogli di bella copia.

Domanda A

Si consideri la ricorrenza $T(n) = T(n-1) + T(n-2)$.

Si dimostri che abbia soluzione del tipo $O(a^n)$ per $a > 1$

DOMANDA A

$$T(n) = T(n-1) + T(n-2) \stackrel{?}{=} O(a^n) \text{ per } a > 1$$
$$T(n) \leq ca^n \quad (c > 0)$$
$$T(n) = T(n-1) + T(n-2) \leq ca^{n-1} + ca^{n-2} = c \frac{a^n}{a} + c \frac{a^n}{a^2} = ca^n \left(\frac{1}{a} + \frac{1}{a^2} \right) \leq ca^n$$
$$\Rightarrow \frac{1}{a} + \frac{1}{a^2} \leq 1 \quad a+1 \leq a^2 \quad a^2 - a - 1 \geq 0$$

$a = \frac{1 + \sqrt{5}}{2}$ (numero d'oro la costante particolare)

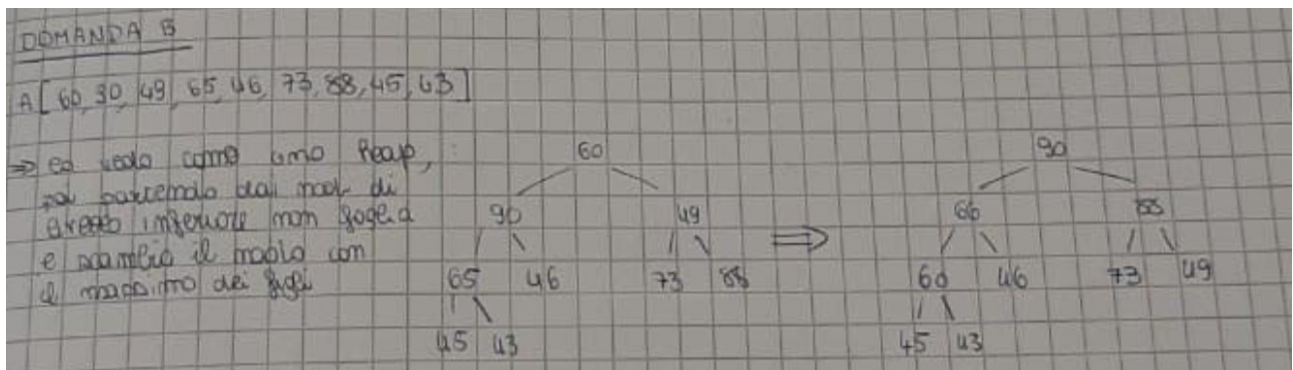
Vogliamo: $T(n) = O(a^n)$; per raggiungerlo non gli stessi, basta cambiare il segno di disuguaglianza

$$\Rightarrow a = \frac{1 + \sqrt{5}}{2}$$

Domanda B

Dare la definizione di max-heap. Dato un array $A[1..12]$ con sequenza di elementi

$[60, 90, 49, 65, 46, 73, 88, 45, 63]$ si indichi il risultato della procedura *BuildMaxHeap* applicata ad A . Si descriva sinteticamente come si procede per arrivare al risultato.



Domanda B (6 punti) Si consideri un insieme di 7 attività $a_i, 1 \leq i \leq 7$, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 4, 2, 3, 7, 8, 11)$$

$$f = (3, 6, 9, 10, 11, 12, 13)$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_2, a_5, a_7\}$.

Esercizio 1

Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno due campi aggiuntivi:

- un campo *pos* che permette di capire la posizione del nodo inserito
- un campo *delta* che permette di capire se dobbiamo ancora mappare le posizioni di altri nodi

Realizzare la procedura di $Insert(T, z)$ che inserisce un nodo z nell'albero e la procedura di stampa $Print(T)$ che dato un albero lo stampa considerando i campi aggiunti.

```

Insert (T, z)
  y = nil
  x = T.root
  if (z.key > x.key) delta = 1 else delta = 0
  while (x != nil)
    y = x
    if (z.key < x.key)
      x = x.left
    else
      x = x.right
    x.pos = x.pos + delta
  z.p = y
  if (y == nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z
  z.pos = delta
  
```

```

Print (T)
  x = T.root
  if (x != nil) and (x.pos > 0)
    PrintPos (x.left)
    if (x.key > 0)
      Print (x.key)
    PrintPos (x.right)
  
```

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```
COMPUTE(a,b)
n <- length(a)
M = -infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  M <- MAX(M,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  M <- MAX(M,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j] * C[i,j+1]
    M <- MAX(M,C[i,j])
return M
```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Algoritmi e Strutture Dati

18 Febbraio 2020

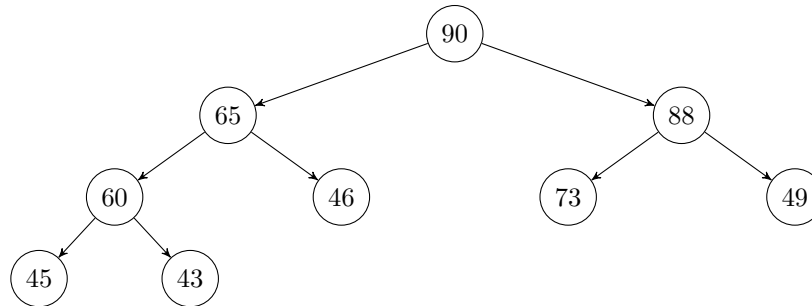
Domande

Domanda A (5 punti) Risolvere la ricorrenza $T(n) = 4T(n/2) + n^2\sqrt{n}$ utilizzando il master theorem.

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 2$, $f(n) = n^2\sqrt{n} = n^{\frac{5}{2}}$. Si osserva che $\log_b a = 2$ quindi $f(n) = \Omega(n^{\log_b a + \epsilon})$ (per $0 < \epsilon \leq \frac{1}{2}$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{2}) = 4\frac{n^2}{4}\sqrt{\frac{n}{2}} = n^2\sqrt{\frac{n}{2}} \leq cn^2\sqrt{n}$ per $c \geq \frac{1}{\sqrt{2}}$ (che è < 1). Quindi $T(n) = \Theta(n^2\sqrt{n})$.

Domanda B (4 punti) Dato l'array $A = [60, 90, 49, 65, 46, 73, 88, 45, 43]$, mostrare in forma di albero, il max-heap prodotto dalla procedura **BuildMaxHeap**.

Soluzione: Per la descrizione del processo di costruzione, si rimanda al libro. Si noti che la procedura **non** consiste in una serie di inserimenti. Il risultato è il seguente:



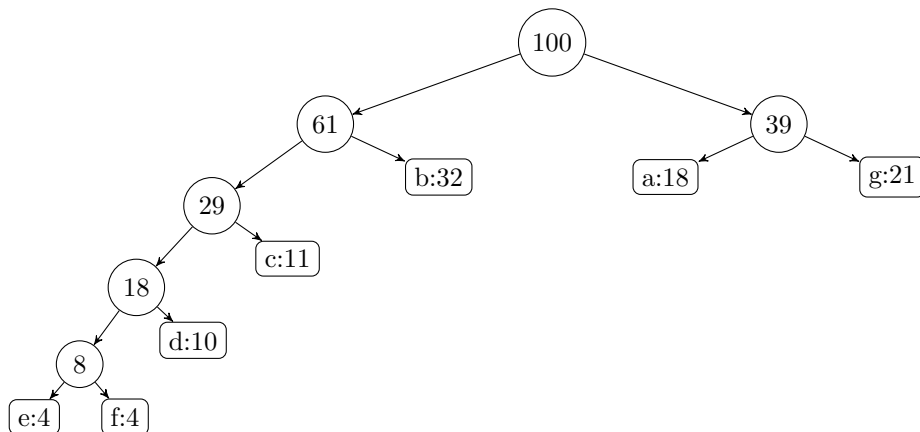
ovvero, in forma di array, $[90, 65, 88, 60, 46, 73, 49, 45, 43]$.

Domanda C (5 punti) Indicare il codice prefisso ottenuto utilizzando l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f, g\}$, supponendo che ogni simbolo appaia con le seguenti frequenze.

a	b	c	d	e	f	g
18	32	11	10	4	4	21

Spiegare il processo di costruzione del codice.

Soluzione: Per il processo di costruzione del codice si rimanda al libro. Il risultato è il seguente:



Esercizi

Esercizio 1 (7 punti) Realizzare una procedura **BST(A)** che dato un array $A[1..n]$ di interi, ordinato in modo crescente, costruisce un albero binario di ricerca di altezza minima che contiene gli elementi di A e ne restituisce la radice. Fornire un'argomentazione che supporti il fatto che l'albero ha altezza minima. Per allocare un nuovo nodo dell'albero si utilizzi una funzione **mknode(k)** che dato un intero k ritorna un nuovo nodo con $x.key=k$ e figlio destro e sinistro $x.left = x.right = nil$. Valutarne la complessità.

Soluzione:

- i. L'implementazione è la seguente:

```

BST(A)
  return BST-rec(A,1,n)

BST-rec(T,A,p,q)
  if p <= q
    m = floor((p+q)/2)
    x=mknode(A[m])
    x.l = BST-rec(A,p,m-1)
    x.r = BST-rec(A,m+1,q)
  else
    x = nil

  return x

```

Si può dimostrare, per induzione su n , che l'altezza dell'albero generato è $\lceil \log_2(n+1) \rceil$, quindi è la minima possibile.

- ii. Si ottiene la ricorrenza $T(n) = 2T(n/2) + \Theta(1)$. Applicando il master theorem si ottiene quindi come costo $T(n) = O(n)$.

Esercizio 2 (10 punti) Si ricordi che data una sequenza $X = x_1 \dots x_k$, si indica con X_i il prefisso $x_1 \dots x_i$. Una sottosequenza di X è $x_{i_1} \dots x_{i_h}$ con $1 \leq i_1 < i_2 < \dots < i_h \leq k$, ovvero è una sequenza ottenuta da X eliminando alcuni elementi. Quando Y è sottosequenza di X si scrive $Y \sqsubseteq X$.

Realizzare un algoritmo che, date due sequenze $X = x_1 \dots x_k$ e $Y = y_1 \dots y_h$ determina una *shortest common supersequence* (SCS) ovvero una sequenza Z , di lunghezza minima, tale che $X \subseteq Z$ e $Y \subseteq Z$. Ad esempio per $X = abf$ e $Y = afgj$ una SCS è $abfgj$.

- i. Dare una caratterizzazione ricorsiva della lunghezza $l_{i,j}$ di una SCS di X_i e Y_j e dedurne un algoritmo;
- ii. valutare la complessità dell'algoritmo.

Soluzione: La caratterizzazione ricorsiva è:

$$l_{i,j} = \begin{cases} i + j & \text{if } i = 0 \text{ or } j = 0 \\ l_{i-1,j-1} + 1 & \text{if } i, j > 0 \text{ e } x_i = y_j \\ \min\{l_{i,j-1}, l_{i-1,j}\} + 1 & \text{if } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Ne segue l'algoritmo che riceve in input le stringhe, nella forma di array di caratteri $X[1, k]$, $Y[1, h]$ e usa una matrice $L[0..k, 0..h]$ dove $L[i, j]$ rappresenta la lunghezza della minima SCS di X_i e Y_j .

```
SCSleng (X,Y,k,h)
  for i=0 to k
    L[i,0] = i
  for j=1 to h
    L[0,j] = j

  for i=1 to k
    for j = 1 to h
      if (X[i] = Y[j])
        L[i,j] = L[i-1,j-1] + 1
      else
        L[i,j] = min (L[i,j-1], L[i-1,j]) + 1

  return L[k,h]
```

La complessità è visto che ci sono due cicli annidati, ripetuti k e h volte, con corpo avente costo costante, $\Theta(hk)$.

Algoritmi e Strutture Dati

26 Giugno 2020

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (7 punti) Definire formalmente la classe $O(f(n))$. Dimostrare che la seguente ricorrenza ha soluzione $T(n) = O(n)$

$$T(n) = \frac{1}{3} T(n-1) + 2n + 1$$

Soluzione: Per la definizione di $O(f(n))$, consultare il libro.

Si deve provare che asintoticamente, per un'opportuna costante $c > 0$

$$T(n) \leq cn$$

Si procede per induzione:

$$\begin{aligned} T(n) &= \frac{1}{3}T(n-1) + 2n + 1 && \text{[per definizione della ricorrenza]} \\ &\leq \frac{1}{3}c(n-1) + 2n + 1 && \text{[per ipotesi induttiva } T(n-1) \leq c(n-1)\text{]} \\ &= \left(\frac{1}{3}c + 2\right)n - \left(\frac{1}{3}c - 1\right) && \text{[assumendo } c \geq 3\text{]} \\ &\leq \left(\frac{1}{3}c + 2\right)n \\ &\leq cn \end{aligned}$$

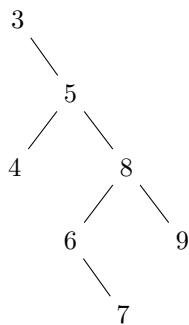
dove l'ultima disuguaglianza vale quando $c \geq 3$, consistentemente con l'assunzione. Risulta dunque dimostrata la tesi.

Domanda B (7 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi:

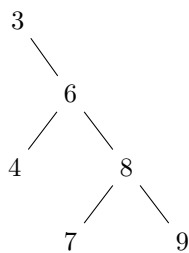
3, 5, 8, 6, 9, 7, 4

Dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione: Per la definizione di albero binario di ricerca e la descrizione delle procedure di inserimento e cancellazione si consulti il libro. Sullo specifico esempio si ottiene, dopo gli inserimenti:



La cancellazione di 5 quindi produce:



Esercizi

Esercizio 1 (9 punti) Realizzare una funzione `avgTree(T)` che dato un albero binario `T` con chiavi numeriche, verifica se, per ogni nodo che abbia discendenti, la chiave del nodo è maggiore o uguale della media delle chiavi dei discendenti e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo `x` ha i campi `x.left`, `x.right` e `x.key`).

Soluzione: L'idea è quella di effettuare una visita dell'albero, in modo ricorsivo, raccogliendo per ogni sottoalbero le seguenti informazioni:

- se è soddisfatta la condizione sulla media richiesta
- la somma delle chiavi
- il numero dei nodi.

Quanto mi trovo su di un nodo, analizzo i sottoalberi sinistro e destro. Quindi, avendo a disposizione la somma delle chiavi ed il numero dei nodi per i due sottoalberi posso verificare la condizione sulla media per il nodo in esame. Lo pseudocodice della soluzione può essere il seguente:


```

avgTree(T)
    v, n, s = avgTreeRec(T.root)
    return v

# avgTreeRec(x):
# verifica se il sottoalbero radicato in x e' un avgTree e ritorna tre valori:
# - un booleano,
# - il numero dei discendenti
# - la somma delle loro chiavi

avgTreeRec(x)

if x = nil
    return true, 0, 0
else
    # ispeziono i sottoalberi sinistro e destro
    vl, nl, sl = avgTreeRec(x.left)
    vr, nr, sr = avgTreeRec(x.right)

    # se non ci sono discendenti (nl+nr =0) oppure la chiave del nodo in
    # esame e' maggiore o uguale della media delle chiavi dei discendenti
    # la condizione e' soddisfatta
    v = (nl+nr =0) or (x.key >= (sl+sr)/(nl+nr))

    # il numero di nodi dell'albero radicato in x e' dato dalla somma del
    # del numero di nodi nel sottoalbero dx e nel sottoalbero sx piu' uno
    # (il nodo x stesso)
    n = nl + nr + 1

    # la somma delle chiavi dell'albero radicato in x e' dato dalla somma del
    # delle chiavi nel sottoalbero dx e in quello sx piu' x.key
    s = sl + sr + x.key

    return v, n, s

```

Si tratta di una visita e quindi la complessità è $\Theta(n)$.

Esercizio 2 (9 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j-1) + c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario alle somme tra numeri interi (escluse quelle tra indici) e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```
COMPUTE(a,b)
n <- length(a)
m = +infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MIN(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] + C[i,j+1]
    m <- MIN(m,C[i,j])
return m
```

- 2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Algoritmi e Strutture Dati

10 Luglio 2020

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti) Definire formalmente la classe $\Theta(g(n))$. Dimostrare le seguenti affermazioni o fornire un controesempio:

- i. se $f(n), f'(n) \in \Theta(g(n))$ allora $f(n) + f'(n) \in \Theta(g(n))$;
- ii. $f(n), f'(n) \in \Theta(g(n))$ allora $f(n) * f'(n) \in \Theta(g(n))$;

Soluzione: Per la definizione di $\Theta(f(n))$, consultare il libro.

Per (i), siano $f(n), f'(n) \in \Theta(g(n))$. Per definizione, esistono $c_1, c_2 > 0$ e n_0 tali che per ogni $n \geq n_0$ vale:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

e, analogamente esistono $c'_1, c'_2 > 0$ e n'_0 tali che per ogni $n \geq n'_0$ vale:

$$0 \leq c'_1 g(n) \leq f'(n) \leq c'_2 g(n)$$

Quindi, per ogni $n \geq \max\{n_0, n'_0\}$ abbiamo:

$$(c_1 + c'_1)g(n) = c_1 g(n) + c'_1 g(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2)g(n)$$

dato che $c_1 + c'_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Per la parte (ii), l'affermazione è falsa. Basta considerare $f(n) = f'(n) = n \in \Theta(n)$, ma ovviamente $f(n) * f'(n) = n^2 \notin \Theta(n)$.

Domanda B (6 punti) Si consideri un insieme di 7 attività $a_i, 1 \leq i \leq 7$, caratterizzate dai seguenti vettori **s** e **f** di tempi di inizio e fine:

$$\mathbf{s} = (1, 4, 2, 3, 7, 8, 11)$$

$$\mathbf{f} = (3, 6, 9, 10, 11, 12, 13).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_2, a_5, a_7\}$.

Esercizi

Esercizio 1 (9 punti) Realizzare una funzione booleana `checkSum(A,B,C,n)` che dati tre array $A[1..n]$, $B[1..n]$ e $C[1..n]$ con valori numerici, verifica se esistono tre indici i, j, k con $1 \leq i, j, k \leq n$ tali che $A[i] + B[j] = C[k]$. Valutarne la complessità.

Soluzione: L'idea consiste nell'ordinare i vettori B e C e a questo punto verificare per ogni elemento $A[i]$ del primo array se ne esiste uno del secondo $B[j]$ la cui somma produce un elemento $C[k]$ del terzo. Il fatto che gli array B e C siano ordinati permette di scorrerli una sola volta per ogni elemento del primo array.

```
checkSum(A,B,C,n):
    # ordina B e C
    Sort(B)
    Sort(C)

    i = 1
    found = false

    while (i <= n) and not found
        # per ogni A[i] verifica se per qualche elemento B[j] vale che
        # A[i]+B[j]=C[k] scorrendo B e C

        j = k = 1
        while (j <= n) and (k <= n) and (A[i] + B[j] <> C[k])
            if (A[i] + B[j] < C[k]):
                j++
            else
                k++
        if (j<=n and k<=n)
            found = true
        else
            i++

    return found
```

L'invariante del ciclo esterno è che se `found` è falsa, allora per ogni $i' < i$, e qualunque siano j' e k' , vale $A[i'] + B[j'] \neq C[k']$. Se invece `found` è vera, allora $i, j, n \leq n$ e $A[i] + B[j] = C[k]$.

Per il ciclo interno, abbiamo che per ogni $j' < j$ e per ogni $k' < k$ vale $A[i] + B[j'] \neq C[k']$. Per il mantenimento dell'invariante, unico fatto da osservare è che quando $A[i] + B[j] < C[k]$ posso incrementare j mantenendo l'invariante. Infatti so che per ogni $j' < j$, dato che B è ordinato, vale $B[j'] \leq B[j]$ e quindi $A[i] + B[j'] \leq A[i] + B[j] < C[k]$. Il ragionamento quando $A[i] + B[j] > C[k]$ e incremento k è duale.

La complessità è $O(n^2)$ dato che ho due cicli annidati. In quello esterno, quando non esco i aumenta, quindi il numero di iterazioni è limitato da n . In quello interno, quando non esco, j oppure k aumentano, quindi il numero di iterazioni è limitato da $2n$. Complessivamente, nel caso peggiore, ho $n * 2n = 2n^2 = \Theta(n^2)$ iterazioni, ciascuna di costo costante.

A questo si somma il costo degli ordinamenti, che però posso realizzare in tempo $\Theta(n \log n)$ di modo che non incida sulla complessità asintotica.

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente

ricorrenza $c(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$c(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ c(i-1, j) \cdot c(i, j+1) \cdot c(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up $\text{COMPUTE_C}(A)$ che, data in input la stringa A restituisca in uscita il valore $c(n, 1)$.
2. Si valuti il numero esatto $T_{CC}(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
COMPUTE_C(A)
n = length(A)
for i=1 to n do
    c[1,i] = a_i
    c[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        c[i,j] = c[i-1,j] * c[i,j+1] * c[i-1,j+1]
return c[n,1]
```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $c[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due operazioni tra interi, e quindi

$$\begin{aligned} T_{CC}(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

Algoritmi e Strutture Dati

27 Agosto 2020

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva che dato un array A e due indici $1 \leq p \leq r \leq A.length$ restituisce la somma degli elementi in $A[p, r]$

```
sum(A,p,r)
  if p < r
    q = (p+r)/2
    return sum(A,p,q) + sum(A,q+1,r)
  else
    return A[p]
```

Dimostrare induttivamente che la funzione è corretta, ovvero che $sum(A, p, r)$ ritorna $\sum_{i=p}^r A[i]$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

Soluzione: Per quanto riguarda la correttezza, procediamo per induzione sul numero n di elementi del sottoarray (ovvero $n = r - p + 1$). Se $n = 1$ l'unico elemento del sottoarray è $A[p]$ e quindi correttamente la funzione ritorna $A[p]$. Se $n > 1$, il sottoarray viene diviso in due parti $A[p, q]$ e $A[q + 1, r]$ di dimensione minore di n , sui quali viene richiamata la funzione. Per ipotesi induttiva $sum(A, p, q) = \sum_{i=p}^q A[i]$ e $sum(A, q + 1, r) = \sum_{i=q+1}^r A[i]$. Quindi la funzione ritorna $\sum_{i=p}^q A[i] + \sum_{i=q+1}^r A[i] = \sum_{i=p}^r A[i]$, come desiderato.

Per quanto riguarda la ricorrenza, indicato ancora con n il numero degli elementi dell'array, nel caso ricorsivo, si effettuano operazioni in tempo costante e quindi si richiama la funzione su due sottoarray la cui dimensione è la metà di quella di partenza. Si ottiene quindi

$$T(n) = 2T(n/2) + c$$

Può essere risolta con il Master Theorem. Utilizzando la notazione del teorema, si deve confrontare la funzione $n^{\log_b a} = n^{\log_2 2} = n^1 = n$ con $f(n) = c$. Chiaramente $f(n) = O(n^{1-\epsilon})$ per ogni $0 < \epsilon < 1$ (infatti $\lim_{n \rightarrow \infty} c/n^{1-\epsilon} = 0$). Quindi siamo nel caso 1 del teorema e concludiamo $T(n) = \Theta(n^{\log_b a}) = \Theta(n)$.

Domanda B (5 punti) (i) Si spieghi brevemente come funzionano le tabelle hash con open addressing. (ii) Data una tabella di dimensione m , la funzione di hash $h(k, i) = (k + 2 * i) \bmod m$ è appropriata? Si motivi la risposta.

Soluzione: Per il funzionamento della tabelle hash con open addressing si rimanda al testo.

La funzione di hash indicata non è appropriata perché, diversamente da quanto richiesto per una funzione di hash, $h(k, 0), h(k, 1), \dots, h(k, m - 1)$ non è, in generale, una permutazione di $0, \dots, m - 1$. Ad esempio, se $m = 10$ e $k = 0$, la sequenza sarà $0, 2, 4, 6, 8, 0, 2, 4, 6, 8$.

Esercizi

Esercizio 1 (9 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , in tempo costante, la media dei valori memorizzati nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `avg(x)` che restituisce la media delle chiavi nel sottoalbero radicato in x e la procedura di inserimento di un nodo `insert(T,z)`.

Soluzione: L'idea è quella di arricchire la struttura facendo in modo che ogni nodo x , oltre ai campi usuali, abbia il campo $x.sum$ che contiene la somma delle chiavi per i nodi del sottoalbero radicato in x e $x.size$ che contiene il numero dei nodi in tale sottoalbero.

A questo punto è immediato realizzare la funzione `avg(x)` di tempo costante

```
avg(x)
  if x <> nil
    return x.sum/x.size
  else
    error
```

Invece per l'inserimento, si modifica la procedura standard, che discende l'albero dalla radice fino alla posizione in cui inserire il nuovo nodo, facendo in modo che tutti i nodi attraversati e che quindi conterranno nel sottoalbero il nuovo nodo, abbiano i campi `sum` e `size` aggiornati.

```
Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x
    x.size++
    x.sum = x.sum + z.key
    if (z.key < x.key)
      x = x.left
    else
      x = x.right

  z.p = y
  if (y <> nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  z.sum = z.key
  z.size = 1
  z.left = z.right = nil
```

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Esercizio 2 (9 punti) Lungo una strada ci sono, in vari punti, n parcheggi liberi e n auto. Un posteggiatore ha il compito di parcheggiare tutte le auto, e lo vuole fare minimizzando lo spostamento totale da fare. Formalmente, dati n valori reali $p_1, p_2, \dots, p_n$ e altri n valori reali $a_1, a_2, \dots, a_n$, che rappresentano

le posizioni lungo la strada rispettivamente di parcheggi e auto, si richiede di assegnare ad ogni auto a_i un parcheggio $p_{h(i)}$ minimizzando la quantità

$$\sum_{i=1}^n |a_i - p_{h(i)}|.$$

1. Si consideri il seguente algoritmo greedy. Si individui la coppia (auto, parcheggio) con la minima differenza. Si assegni quell'auto a quel parcheggio. Si ripeta con le auto e i parcheggi restanti fino a quando tutte le auto sono parcheggiate. Dimostrare che questo algoritmo non è corretto, esibendo un controesempio.
2. Si consideri il seguente algoritmo greedy. Si assuma che i valori $p_1, p_2, \dots, p_n$ e $a_1, a_2, \dots, a_n$ siano ordinati in modo non decrescente. Si produca l'assegnazione $(a_1, p_1), (a_2, p_2), \dots, (a_n, p_n)$. Dimostrare la correttezza di questo algoritmo per il caso $n = 2$.

Soluzione:

1. Si consideri il seguente input:

$$p_1 = 5, p_2 = 10 \quad \text{e} \quad a_1 = 9, a_2 = 14.$$

L'algoritmo produce l'assegnazione $(a_1, p_2), (a_2, p_1)$, che ha costo $1 + 9 = 10$, mentre l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $4 + 4 = 8$.

2. Ci sono vari casi possibili:

(a) Caso $a_1 \leq p_1 \leq p_2 \leq a_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + a_2 - p_2 = (a_2 - a_1) - (p_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (a_2 - a_1) + (p_2 - p_1)$; siccome $p_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(b) Caso $a_1 \leq p_1 \leq a_2 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) - (a_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (p_2 - a_1) + (a_2 - p_1)$; siccome $a_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(c) Caso $a_1 \leq a_2 \leq p_1 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) + (p_1 - a_2)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + p_1 - a_2 = (p_2 - a_1) + (p_1 - a_2)$, uguale a quello precedente

Tutti gli altri casi sono simmetrici e si dimostrano nella stessa maniera.

Algoritmi e Strutture Dati

10 Settembre 2020

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
exp(n)
  if n = 0
    return 0
  else
    return exp(n-1) + exp(n-1) + 1
```

Dimostrare induttivamente che la funzione calcola il valore $2^n - 1$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Omega(2^n)$. È anche $O(2^n)$? Motivare le risposte.

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione su n . Se $n = 0$, la funzione ritorna 0 che è in effetti $2^0 - 1 = 1 - 1 = 0$. Se $n > 0$, allora per ipotesi induttiva, $\text{exp}(n-1) = 2^{n-1} - 1$. Quindi $\text{exp}(n) = \text{exp}(n-1) + \text{exp}(n-1) + 1 = 2^{n-1} - 1 + 2^{n-1} - 1 + 1 = 2 \cdot 2^{n-1} - 1 = 2^n - 1$, come desiderato.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, si effettuano due chiamate ricorsive con argomento $n-1$ e tre somme (operazioni di tempo costante) si ottiene quindi

$$T(n) = 2T(n-1) + c$$

È facile dimostrare con il metodo di sostituzione che $T(n) = \Omega(2^n)$, ovvero che esistono $d > 0$ e n_0 tali che $T(n) \geq d2^n$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= 2T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq 2d2^{n-1} + c && \text{[ipotesi induttiva]} \\ &= d2^n + c \\ &\geq d2^n \end{aligned}$$

qualunque sia $d > 0$ e n .

Vale anche $T(n) = O(2^n)$. Tuttavia, il tentativo di provare direttamente per induzione che $d > 0$ e n_0 tali che $T(n) \leq d2^n$ per un'opportuna costante $d > 0$ e $n \geq n_0$ fallisce. Si riesce invece a dimostrare che $T(n) \leq d(2^n - 1)$. Infatti:

$$\begin{aligned} T(n) &= 2T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq 2d(2^{n-1} - 1) + c && \text{[ipotesi induttiva]} \\ &= d2^n - 2d + c \\ &= d(2^n - 1) - d + c \\ &\leq d2^n \end{aligned}$$

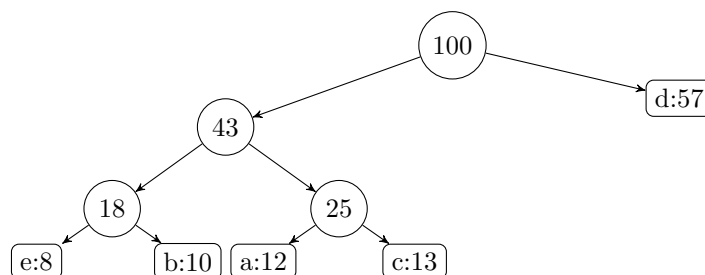
dove l'ultima disuguaglianza vale se $d \geq c$. Quindi $T(n) = O(2^n - 1) = O(2^n)$.

Domanda B (5 punti) Indicare, in forma di albero binario, il codice prefisso ottenuto tramite l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e\}$, supponendo che ogni simbolo appaia con le seguenti frequenze:

a	b	c	d	e
12	10	13	57	8

Spiegare il processo di costruzione del codice.

Soluzione: Per la descrizione del processo di costruzione, si rimanda al libro. Il risultato è il seguente:



Esercizi

Esercizio 1 (9 punti) Realizzare una funzione $append(T, x, z)$ che dato un albero binario di ricerca T , un suo nodo *foglia* x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna `nil`. (Assumere che i nodi dell'albero abbiano gli usuali campi $x.left$, $x.right$, $x.p$, $x.key$ e la radice sia $T.root$.) Valutarne la complessità.

Soluzione: Affinchè z possa essere inserito come figlio destro o sinistro di x , occorre in primo luogo che la chiave di z sia $\leq$ delle chiavi degli antenati di z che hanno x e quindi z nel sottoalbero sinistro e $\geq$ delle chiavi degli antenati di x che hanno x nel sottoalbero destro.

Quindi risalendo tali antenati, si verifica che la condizione sia soddisfatta. Se questo accade e $z.key$ è $\leq x.key$ potrà essere inserito come figlio sinistro di x , altrimenti come figlio destro. Altrimenti z non può essere figlio di x e si ritorna *nil*.

Lo pseudocodice può essere il seguente:

```

append(T,x,z)

# verifica che z.key sia <= delle chiavi degli antenati di x
# per i quali x e' nel sottoalbero dx e
y = x

while (y.p <> nil) and
    ( (y == y.p.left   and  z.key <= y.p.key) or
      (y == y.p.right  and  z.key >= y.p.key) )
    y = y.p

# se siamo arrivati alla radice, significa che la condizione e' soddisfatta
if (y.p == nil)
    # e z viene inserito come figlio dx o sx
    z.p = x
    if (z.key <= x.key)
        x.left = z
    else
        x.right = z

    return T
else
    # altrimenti si ritorna nil
    return nil

```

La complessità è chiaramente $O(h)$, dove h è l'altezza dell'albero. Infatti, il ciclo while effettua un numero di iterazioni pari al numero di antenati di x , e quindi limitato dall'altezza dell'albero. E ciascuna iterazione ha costo costante.

Esercizio 2 (9 punti) Sia $n > 0$ un intero. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 1 & \text{se } i = j, \\ 2 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) \cdot M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

1. Scrivere una coppia di algoritmi INIT_M(n) e REC_M(i, j) per il calcolo memoizzato di $M(1, n)$.
2. Calcolare il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Soluzione:

1. Pseudocodice:

```

INIT_M(n)
if n=1 then return 1
if n=2 then return 2
for i=1 to n-1 do
    M[i,i] = 1
    M[i,i+1] = 2
M[n,n] = 1
for i=1 to n-2 do
    for j=i+2 to n do
        M[i,j] = 0
return REC_M(1,n)

REC_M(i,j)
if M[i,j] = 0 then
    M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) * REC_M(i,j-1)
return M[i,j]

```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 = 2 \sum_{i=1}^{n-2} n - i - 1 = 2 \sum_{k=1}^{n-2} k = (n-2)(n-1)$$

Algoritmi e Strutture Dati

8 Febbraio 2021

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti) Si consideri la funzione ricorsiva `search(A,p,r,k)` che dato un array A , ordinato in modo crescente, un valore k e due indici p, q con $1 \leq p \leq r \leq A.length$ restituisce un indice i tale che $p \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0.

```
search(A,p,r,k)
  if p <= r
    q = (p+r)/2
    if A[q] = k
      return q
    elseif A[q] < k
      return search(A,q+1,r,k)
    else
      return search(A,p,q-1,k)
  else
    return 0
```

Dimostrare induttivamente che la funzione è corretta. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

Soluzione: La prova di correttezza avviene per induzione sulla lunghezza $l = r - p + 1$ del sottoarray $A[p..r]$ di interesse. Se $l = 0$, ovvero $p > r$, la funzione ritorna correttamente 0, dato che non ci sono elementi nel sottoarray e quindi non ci possono essere elementi $= k$. Se invece $l > 0$ si calcola $q = \lfloor (p+r)/2 \rfloor$ e si distinguono tre casi:

- se $A[q] = k$ si ritorna correttamente k
- se $A[q] < k$, dato che l'array è ordinato certamente $A[j] \leq A[q] < k$ per $p \leq j \leq q$. Quindi il valore k può trovarsi solo nel sottoarray $A[q+1, r]$. La chiamata `search(A, q+1, r, k)`, dato che il sottoarray ha lunghezza minore di l , per ipotesi induttiva restituisce un indice i tale che $q+1 \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. Per l'osservazione precedente, questo è il valore corretto da restituire anche per `search(A, p, r, k)` e si conclude.
- se $A[q] > k$ si ragiona in modo duale rispetto al caso precedente.

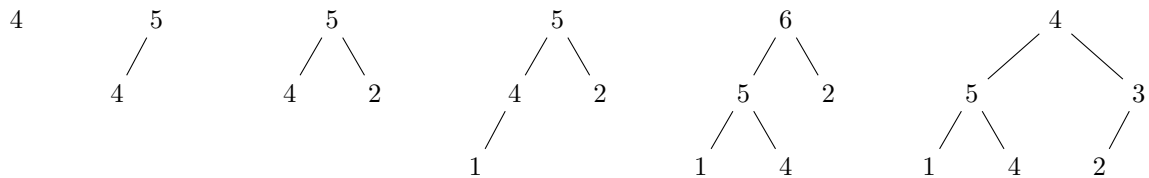
Per quanto riguarda la ricorrenza, ignorando i casi base, dato che la funzione ricorre su di un array la cui dimensione è la metà di quello originale, si ottiene:

$$T(n) = T(n/2) + c$$

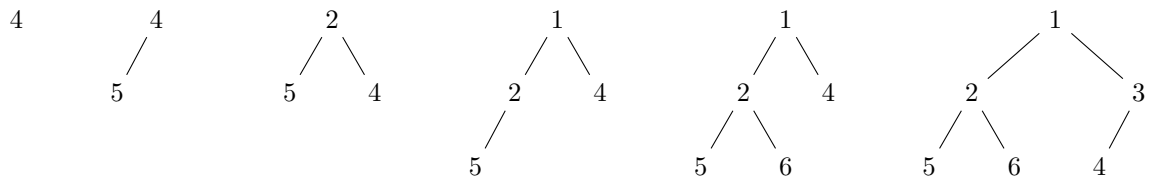
Rispetto allo schema standard del master theorem ho $a = 1$, $b = 2$ e $f(n) = c$. Ho dunque che $f(n) = 1 = \Theta(n^{\log_2 1}) = \Theta(n^0) = \Theta(1)$. Pertanto si conclude che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

Domanda B (5 punti) Si dia la definizione della struttura dati max-heap. Si supponga di inserire in un max-heap inizialmente vuoto, in successione, gli elementi 4, 5, 2, 1, 6, 3. Fornire la rappresentazione (ad albero) del max-heap così ottenuto. Si motivi la risposta mostrando la sequenza di max-heap ottenuti.

Soluzione: Per la definizione di max-heap e la procedura di inserzione di un elemento si rimanda al testo. La sequenza di max-heap ottenuta con l'inserimento degli elementi nell'ordine indicato è:



mentre invece la sequenza di min-heap è:



Esercizi

Esercizio 1 (9 punti) Realizzare una funzione `intersect(A1,A2,n)` che dati due array di interi **A1** e **A2**, organizzati a min-heap, con capacità n , restituisce un nuovo array **A**, ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in **A1** e **A2**. Nel caso gli array contengano più occorrenze dello stesso valore v , l'intersezione mantiene il numero minimo di occorrenze di v (ad es. se **A1** contiene i valori 1,2,2,2 e **A2** contiene i valori 1,1,2,2 allora **A** conterrà 1,2,2). Valutarne la complessità.

Soluzione: Per costruire l'intersezione, si procede nel modo seguente. Posso estrarre gli elementi di ciascun min-heap, in ordine crescente, semplicemente con una sequenza di estrazioni di minimo, ciascuna delle quali ha costo logaritmico. Se da **A1** estraggo v_1 e da **A2** estraggo v_2 , se $v_1 = v_2$, inserisco il valore comune nell'intersezione. Se invece $v_1 < v_2$, certamente v_1 non è nell'intersezione, dato che tutti gli elementi rimanenti in **A2** sono $\geq v_2$, quindi posso scartare v_1 , estrarre un nuovo elemento da **A1** e continuare. Se $v_1 > v_2$ procedo dualmente.

In questo modo ottengo gli elementi dell'intersezione in ordine crescente. L'ultima osservazione è che posso inserirli in questo modo in **A**, che risulterà un array crescente, che è un caso particolare di min-heap.

Lo pseudocodice può essere il seguente:

```

intersect(A1,A2,n)
  allocate A[1..n]
  i=0 // ultimo elemento occupato in A
  while (A1.heapsize > 0) and (A2.heapsize > 0)
    v1 = Min(A1)
    v2 = Min(A2)
    if (v1 = v2)
      i++
      A[i]=1
      ExtractMin(A1)
      ExtractMin(A2)
    elseif (v1 < v2)
      ExtractMin(A1)
    else
      ExtractMin(A2)

  A.heapsize=i
  return A

```

La complessità di ogni `ExtractMin` è $O(\log n)$ mentre `Min` ha costo costante. Il numero di estrazioni è chiaramente limitato da $2n$ e questo porta a dedurre che il costo complessivo è $O(n \log n)$.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j - 1) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = +infinite
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MIN(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] * C[i,j+1]
    m <- MIN(m,C[i,j])
return m

```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Algoritmi e Strutture Dati

18 Giugno 2021

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti)

Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
val(n)
  if n = 0
    return 0
  else
    return val(n-1) + n + 1
```

Dimostrare induttivamente che la funzione calcola il valore $(n+3)n/2$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Theta(n)$. Motivare le risposte.

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione su n . Se $n = 0$, la funzione ritorna 0 che è in effetti $(0+3)0/2 = 0/2 = 0$. Se $n > 0$, allora per ipotesi induttiva, $val(n-1) = (n+2)(n-1)/2$. Quindi $val(n) = val(n-1) + n + 1 = (n+2)(n-1)/2 + n + 1 = (n^2 + n - 2 + 2n + 2)/2 = (n^2 + 3n)/2 = (n+3)n/2$, come desiderato.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, si effettuano una chiamata ricorsiva con argomento $n-1$ e una somma (operazione di tempo costante) si ottiene quindi

$$T(n) = T(n-1) + c$$

È facile dimostrare con il metodo di sostituzione che $T(n) = \Theta(n)$. Dimostriamo, ad esempio, che $T(n) = \Omega(n)$, ovvero che esistono $d > 0$ e n_0 tali che $T(n) \geq dn$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq d(n-1) + c && \text{[ipotesi induttiva]} \\ &= dn - d + c \\ &\geq dn \end{aligned}$$

per una costante d tale che $0 < d \leq c$ e qualunque n .

Vale anche $T(n) = O(n)$, ovvero esistono $d > 0$ e n_0 tali che $T(n) \leq dn$ per un'opportuna costante $d > 0$ e $n \geq n_0$. Infatti:

$$\begin{aligned} T(n) &= T(n-1) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq d(n-1) + c && \text{[ipotesi induttiva]} \\ &= dn - d + c \\ &\leq dn \end{aligned}$$

dove l'ultima disuguaglianza vale per una costante $d \geq c$ e qualunque sia n .

Domanda B (6 punti) Si consideri un insieme di 8 attività $a_i, 1 \leq i \leq 8$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (1, 1, 2, 4, 2, 5, 6, 9)$$

$$\mathbf{f} = (3, 4, 5, 6, 7, 9, 10, 12).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_4, a_7\}$.

Esercizi

Esercizio 1 (9 punti) Scrivere una funzione $\text{Anc}(\mathbf{T}, \mathbf{k1}, \mathbf{k2})$ che dato un albero binario di ricerca $\mathbf{T}$ nel quale tutte le chiavi sono distinte, e due chiavi $\mathbf{k1}, \mathbf{k2}$ presenti in $\mathbf{T}$, verifica se il nodo contenente $\mathbf{k1}$ è antenato del nodo contenente $\mathbf{k2}$. Valutare la complessità della funzione.

Soluzione: È sufficiente osservare che, dato che le chiavi sono uniche e $\mathbf{k1}, \mathbf{k2}$ sono certamente contenute in $\mathbf{T}$, il nodo che contiene $\mathbf{k1}$ è antenato di quello che contiene $\mathbf{k2}$ se e solo se cercando $\mathbf{k2}$ a partire dalla radice di $\mathbf{T}$ incontro la chiave $\mathbf{k1}$. Si assume che un nodo sia antenato di sé stesso.

La funzione può dunque essere realizzata come una semplice variante della ricerca negli alberi binari di ricerca.

```
Anc(T, k1, k2)
    x = T.root

    while (x.key <> k1) and (x.key <> k2)
        if (k2 < x.key)
            x = x.left
        else
            x = x.right

    return (x.key == k1)
```

Se l'albero ha altezza h , nel caso peggiore non trovo $\mathbf{k1}$ e la chiave $\mathbf{k2}$ è una foglia a profondità h , che quindi raggiunge in h iterazioni. Pertanto la complessità è $O(h)$.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .

2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```
COMPUTE(a,b)
n <- length(a)
M = -infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  M <- MAX(M,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  M <- MAX(M,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j] * C[i,j+1]
    M <- MAX(M,C[i,j])
return M
```

- 2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n - 1 - i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Algoritmi e Strutture Dati

6 luglio 2021

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (7 punti) Definire formalmente la classe $\Theta(g(n))$. Dimostrare le seguenti affermazioni o fornire un controesempio:

- i. se $f(n) \in \Theta(g(n))$ e $f'(n) \in O(g(n))$ allora $f(n) + f'(n) \in \Theta(g(n))$;
- ii. se $f(n) \in \Theta(g(n))$ e $f'(n) \in O(g(n))$ allora $f(n) - f'(n) \in \Theta(g(n))$;

Soluzione: Per la definizione di $\Theta(f(n))$, consultare il libro.

Per (i), sia $f(n) \in \Theta(g(n))$. Per definizione, esistono $c_1, c_2 > 0$ e n_0 tali che per ogni $n \geq n_0$ vale:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Inoltre, dato che $f'(n) \in O(g(n))$ esistono $c'_2 > 0$ e n'_0 tali che per ogni $n \geq n'_0$ vale:

$$0 \leq f'(n) \leq c'_2 g(n)$$

Quindi, per ogni $n \geq \max\{n_0, n'_0\}$ abbiamo:

$$0 \leq c_1 g(n) \leq f(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2) g(n)$$

dato che $c_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Quindi, essendo $k > 0$, per ogni $n \geq \max n_0$ abbiamo:

$$(c_1 + c'_1) g(n) = c_1 g(n) + c'_1 g(n) \leq f(n) + f'(n) \leq c_2 g(n) + c'_2 g(n) = (c_2 + c'_2) g(n)$$

dato che $c_1 + c'_1, c_2 + c'_2 > 0$ questo conclude la prova che $f(n) + f'(n) \in \Theta(g(n))$.

Per la parte (ii), l'affermazione è falsa. Basta considerare $f(n) = f'(n) = n \in \Theta(n)$ e quindi in $O(n)$, ma ovviamente $f(n) - f'(n) \notin \Omega(n)$ e quindi $\notin \Theta(n)$.

Domanda B (7 punti) Scrivere una funzione `toTree(A)` che dato un array **A** organizzato a max-heap (dimensione `A.heapSize`), lo trasforma in un albero binario realizzato con strutture linked, ancora organizzato a max-heap e ritorna la radice di tale albero. Il nuovo albero è costituito da nodi `x` con i campi `x.p` (parent), `x.k` (chiave), `x.l` e `x.r` (figlio sinistro e figlio destro). Per allocare un nuovo nodo si assuma di avere a disposizione un costruttore `node()`. Valutare la complessità.

Soluzione: L'idea è semplicemente quella di visitare l'albero, memorizzato come array, in profondità, e riprodurlo nella struttura linked.

```

toTree(A)
    T.root = toTreeRec(A,1,nil)
    return T

// toTreeRec(A,i,x):
// dato un max-heap A ritorna la radice di un albero, copia "linked" del
// sottoalbero di A radicato in i, l'argomento x e' il nodo padre
toTreeRec(A,i,x)
    if i <= A.heapSize
        y = node()                // crea un nuovo nodo
        y.key = A[i]              // la chiave e' A[i]
        y.p = x                   // il parent e' x
        left = 2*i                // costruisce i sottoalberi sx e dx,
        right = 2*i+1             // che saranno figli sx e dx di y
        y.l = toTreeRec(A,left, y)
        y.r = toTreeRec(A,right,y)
    else
        return nil

```

Si tratta di una visita dell'albero e quindi la complessità è $\Theta(n)$ dove n è il numero di elementi del max-heap.

Esercizi

Esercizio 1 (8 punti) Realizzare una funzione `missing(A,n)` che dato un array `A[1..n]` che contiene interi nell'intervallo `[1,n]` verifica se esiste qualche valore $v \in [1,n]$ che non compare in `A`, ovvero tale che $A[i] \neq v$ per ogni $i \in [1,n]$. Valutare la complessità.

Soluzione: Si osserva che in `A[1..n]` possono mancare dei valori se e solo se ci sono elementi duplicati.

Per verificare la presenza di duplicati, si scorre l'array `A` e si usa un array di booleani `B[1..n]` per ricordare i valori già incontrati. Più precisamente `B[v]` indica se è stato o meno incontrato il valore v nella parte di array `A` esplorata. Ci sarà un duplicato (e quindi un valore assente) solo se durante l'esplorazione si incontra un valore v che era già stato incontrato prima e quindi con `B[v]=true`.

```

missing(A,n)
    alloca B[1..n] array di booleani

    for i=1 to n
        B[i]=false

    i=1
    while (i<=n) and not B[A[i]]
        B[A[i]] = true
        i++

    return i<=n

```

L'invariante del ciclo è

- $A[1, i - 1]$ non contiene duplicati e

- $\forall v \in [1, n]. B[v] = \text{true}$ se e solo se $\forall j \in [1, i-1]. A[j] \neq v$.

Quindi, se si esce perché $i = n + 1$, significa che $A[1, n]$ non contiene duplicati e quindi non ci sono valori assenti.

Se invece si esce con $i \leq n$, significa che $B[A[i]] = \text{true}$. Pertanto deve esistere $j < i$ tale che $A[j] = A[i]$, ovvero c'è un duplicato e quindi necessariamente un valore assente.

La complessità è chiaramente $\Theta(n)$, dato che il ciclo di inizializzazione di B costa n , e il while fa al massimo n iterazioni, ciascuna di costo costante.

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Soluzione:

1. Pseudocodice:

```
INIT_L(X)
n <- length(X)
if n = 1 then return 1
if n = 2 then return 2
for i=1 to n-1 do
    L[i,i] <- 1
    L[i,i+1] <- 2
L[n,n] <- 1
for i=1 to n-2 do
    for j=i+2 to n do
        L[i,j] <- 0
return REC_L(X,1,n)

REC_L(X,i,j)
if L[i,j] = 0 then
    if x_i = x_j then L[i,j] <- 2 + REC_L(X,i+1,j-1)
    else for k=i to j-1 do
        L[i,j] <- L[i,j] + REC_L(X,i,k) + REC_L(X,k+1,j)
return L[i,j]
```

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

$(1, n), (2, n-1), \dots, (k, n-k+1)$, ognuno associato a un costo unitario. Ci sono al più $\lfloor n/2 \rfloor$ di queste chiamate, e quindi $T_{\text{best}}(n) = O(n)$.

Algoritmi e Strutture Dati

31 agosto 2021

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano i fogli di bella copia.

Domande

Domanda A (7 punti) Si consideri la funzione ricorsiva `search(A, p, r, k)` che dato un array A , ordinato in modo decrescente, un valore k e due indici p, q con $1 \leq p \leq r \leq A.length$ restituisce un indice i tale che $p \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0.

```
search(A, p, r, k)
    if p <= r
        q = (p+r)/2
        if A[q] = k
            return q
        elseif A[q] > k
            return search(A, q+1, r, k)
        else
            return search(A, p, q-1, k)
    else
        return 0
```

Dimostrare induttivamente che la funzione è corretta. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

Soluzione: La prova di correttezza avviene per induzione sulla lunghezza $l = r - p + 1$ del sottoarray $A[p..r]$ di interesse. Se $l = 0$, ovvero $p > r$, la funzione ritorna correttamente 0, dato che non ci sono elementi nel sottoarray e quindi non ci possono essere elementi $= k$. Se invece $l > 0$ si calcola $q = \lfloor (p+r)/2 \rfloor$ e si distinguono tre casi:

- se $A[q] = k$ si ritorna correttamente k
- se $A[q] > k$, dato che l'array è ordinato in modo decrescente certamente $A[j] \geq A[q] > k$ per $p \leq j \leq q$. Quindi il valore k può trovarsi solo nel sottoarray $A[q+1, r]$. La chiamata `search(A, q+1, r, k)`, dato che il sottoarray ha lunghezza minore di l , per ipotesi induttiva restituisce un indice i tale che $q+1 \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. Per l'osservazione precedente, questo è il valore corretto da restituire anche per `search(A, p, r, k)` e si conclude.
- se $A[q] < k$ si ragiona in modo duale rispetto al caso precedente.

Per quanto riguarda la ricorrenza, ignorando i casi base, dato che la funzione ricorre su di un array la cui dimensione è la metà di quello originale, si ottiene:

$$T(n) = T(n/2) + c$$

Rispetto allo schema standard del master theorem ho $a = 1$, $b = 2$ e $f(n) = c$. Ho dunque che $f(n) = 1 = \Theta(n^{\log_2 1}) = n^0 = 1$. Pertanto si conclude che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

Domanda B (6 punti) Si consideri una tabella hash di dimensione $m = 8$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + 2(k \bmod (m-2))$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 13, 29, 19, 27, 8.

Soluzione: Si ottiene

0	29
1	-
2	27
3	19
4	-
5	13
6	-
7	8

Esercizi

Esercizio 1 (9 punti) Si consideri un albero binario T , i cui nodi x hanno i campi $x.l$, $x.r$, $x.p$ che rappresentano il figlio sinistro, il figlio destro e il padre, rispettivamente. Un *cammino* è una sequenza di nodi $x_0, x_1, \dots, x_n$ tale che per ogni $i = 0, \dots, n-1$ vale $x_{i+1}.p = x_i$. Il cammino è detto *nil-terminated* se $x_n.l = \text{nil}$ oppure $x_n.r = \text{nil}$. Dato un certo k , diciamo che l'albero è k -bilanciato se tutti i cammini nil-terminated che iniziano dalla radice hanno lunghezze che differiscono al più di k . Scrivere una funzione $\text{bal}(T, k)$ che dato in input l'albero T e un valore k verifica se T è k -bilanciato e ritorna un corrispondente valore booleano. Valutarne la complessità.

Soluzione: La soluzione può basarsi su di una funzione ricorsiva che calcola per un nodo x la lunghezza massima e minima dei cammini che iniziano da x e sono nil-terminated

```
bal(T,k)    // verifica se l'albero radicato in x e' k-bilanciato,
            // ovvero i cammini dalla radice terminati da nil hanno
            // lunghezze che differiscono al piu di k

            // si basa su di una funzione ricorsiva che calcola la
            // lunghezza minima e massima dei cammini nil-terminated
            // da un certo nodo.
min,max = nilTerm(T.root)

return max-min <= k

nilTerm(x)
if x = nil
    return (0,0)
else
    (left_min,left_max) = nilTerm(x.left)
    (right_min,r_right_max) = nilTerm(x.right)
    return min(left_min, right_min) + 1, max(left_max,right_max) + 1
```

Per quanto riguarda la complessità si osservi che $T(n) = c + T(k) + T(n - k - 1)$ per un'opportuna costante c e quindi, la complessità è $O(n)$.

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ z(i-1, j) \cdot z(i, j+1) \cdot z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up $Z(A)$ che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$.
2. Si valuti il numero esatto $T_Z(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all’interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
Z(A)
n = length(A)
for i=1 to n do
    z[1,i] = a_i
    z[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        z[i,j] = z[i-1,j] * z[i,j+1] * z[i-1,j+1]
return z[n,1]
```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $z[1, n]$.

2. Ogni iterazione del doppio ciclo dell’algoritmo esegue due moltiplicazioni tra interi, e quindi

$$\begin{aligned} T_Z(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l’algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

Algoritmi e Strutture Dati

24 settembre 2021

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano i fogli di bella copia.

Domande

Domanda A (6 punti) Data la ricorrenza $T(n) = 6T(n/3) + (n-1)^2$, trovare la soluzione asintotica.

Soluzione: Si usa il master theorem con $a = 6$, $b = 3$, $f(n) = (n-1)^2$. Si deve confrontare $f(n)$ con $n^{\log_b a} = n^{\log_3 6}$ ed essendo che $\log_3 6 < 2$ per qualunque $0 < \epsilon < 2 - \log_3 6$ si vede facilmente che $f(n) = \Omega(n^{\log_3 6 + \epsilon})$.

Per concludere che $T(n) = \Theta(f(n)) = \Theta(n^2)$ usando il caso 3 del Master Theorem occorre dimostrare la regolarità di $f(n)$, ovvero che $af(n/b) < kf(n)$ per $0 < k < 1$, asintoticamente. Si imposta

$$af(n/b) = 6(n/3 - 1)^2 < k(n-1)^2$$

e si osserva

$$6(n/3 - 1)^2 = 6((n-6)/3)^2 < 6((n-2)/3)^2 = 6/9(n-2)^2 = 2/3(n-2)^2$$

per cui si può scegliere $k = 2/3 < 1$ e n qualunque.

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *toro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	<i>t</i>	<i>o</i>	<i>r</i>	<i>o</i>
	0	0	0	0
<i>a</i>	0	0	0	0
<i>r</i>	0	0	0	1
<i>m</i>	0	0	0	1
<i>o</i>	0	0	1	2

La lunghezza della longest common subsequence tra *armo* e *toro* è quindi 2.

Esercizi

Esercizio 1 (10 punti) Dare la definizione di albero binario di ricerca. Scrivere una funzione `conta(T, a, b)` che dato un albero binario di ricerca `T` e due valori `a` e `b`, ritorna il numero di nodi `x` tali che `x.key` è in $[a, b]$. Mostrarne la correttezza e valutarne la complessità.

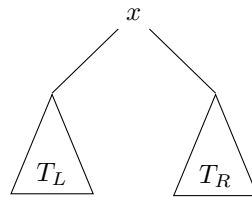
Soluzione: L'idea è quella di realizzare una funzione ricorsiva `contaRec(x,a,b)`, che dato un nodo x , restituisce il numero di nodi nel sottoalbero radicato in x con chiave nell'intervallo $[a,b]$. Quindi la si applica alla radice dell'albero.

```
conta(T,a,b)
    return contaRec(T,root, a,b)

contaRec(x, a, b)
    if x <> nil
        if b < x.key
            return contaRec(x.left)
        else if x.key < a
            return contaRec(x.right)
        else
            return 1 + contaRec(x.right) + contaRec(x.left)
```

La correttezza della funzione ricorsiva può essere dimostrata per induzione sull'altezza h del sottoalbero radicato in x

- ($h = 0$) In questo caso, $x = nil$, il sottoalbero è vuoto e quindi correttamente si ritorna 0.
- ($h > 0$) In questo caso l'albero ha la forma



Si distinguono tre casi:

- se $b < x.key$, sicuramente, per le proprietà degli ABR, tutti i nodi nel sottoalbero T_R hanno chiave maggiore di $x.key$ e quindi di b , e pertanto non sono nell'intervallo. Dunque si ritorna il numero di nodi in $[a,b]$ nel sottoalbero T_L , che per ipotesi induttiva è fornito da `contaRec(x.left)`.
- $x.key < a$ si ragiona in modo duale rispetto al caso precedente
- altrimenti, $x.key \in [a,b]$ e i nodi con chiave in $[a,b]$ possono trovarsi sia nel sottoalbero sinistro che nel sottoalbero destro. Si ritorna dunque $1 + \text{contaRec}(x.right) + \text{contaRec}(x.left)$, dove le chiamate `contaRec(x.right)` e `contaRec(x.left)` ritornano, per ipotesi induttiva, i nodi in $[a,b]$ nei sottoalberi T_R e T_L .

Nel caso peggiore, ad esempio quando tutti i nodi hanno chiave in $[a,b]$, la funzione esegue una visita completa dell'albero e quindi la complessità è $O(n)$ dove n è il numero dei nodi.

Esercizio 2 (9 punti) Si consideri un file definito sull'alfabeto $\Sigma = \{a, b, c\}$, con frequenze $f(a), f(b), f(c)$. Per ognuna delle seguenti codifiche si determini, se esiste, un opportuno assegnamento di valori alle 3 frequenze $f(a), f(b), f(c)$ per cui l'algoritmo di Huffman restituisce tale codifica, oppure si argomenti che tale codifica non è mai ottenibile.

1. $e(a) = 0, e(b) = 10, e(c) = 11$

2. $e(a) = 1, e(b) = 0, e(c) = 11$
3. $e(a) = 10, e(b) = 01, e(c) = 00$

Soluzione:

1. Questa codifica viene restituita dall'algoritmo di Huffman quando $f(b), f(c) < f(a)$: in questo caso i nodi associati a b e c vengono uniti creando un nuovo nodo interno, che poi viene unito al nodo associato ad a . Quindi, per esempio, $f(a) = 40, f(b) = 25, f(c) = 35$.
2. La codeword $e(a) = 1$ è un prefisso della codeword $e(c) = 11$, cioè questa codifica non è libera da prefissi, e quindi non è una codifica di Huffman.
3. L'albero associato a questa codifica non è pieno perché c'è un nodo interno con un solo figlio (quello nel cammino che porta alla foglia associata al carattere a). Quindi questa codifica non è ottima e quindi non è una codifica di Huffman.

Algoritmi e Strutture Dati

24 gennaio 2022

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un array A di interi e due indici $1 \leq p \leq r \leq A.length$:

```
Minimum(A,p,r)
    if p=r
        return A[p]
    else
        q = (p+r)/2
        return min(Minimum(A,p,q), Minimum(A,q+1,r))
```

Dimostrare induttivamente che la funzione calcola il minimo del sottoarray $A[p..r]$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Theta(n)$, dove n indica la lunghezza del sottoarray. Motivare le risposte.

Soluzione: Per quanto riguarda la funzione calcolata, procediamo per induzione sulla lunghezza $n = r - p + 1$ del sottoarray $A[p..q]$. Se $n = 1$, il sottoarray di interesse contiene un solo elemento $A[p]$ che è correttamente l'elemento ritornato. Se invece $n > 1$, le due chiamate ricorsive $Minimum(A, p, q - 1)$ e $Minimum(A, q, r)$, su sottoarray di dimensione $< n$ ritornano il minimi in $A[p..q]$ e $A[q + 1, r]$. Viene ritornato il valore minimimo tra i due, che srà correttamente il minimo valore in $A[p..r]$.

Per quanto riguarda la ricorrenza, nel caso ricorsivo, se n è la lunghezza del sottoarray di interesse, si effettuano due chiamate ricorsive su sottorray di dimensione $n/2$ e un'operazione di minimo (operazione di tempo costante) si ottiene quindi

$$T(n) = 2T(n/2) + c$$

Per risolvere la ricorrenza possiamo applicare il Master Theorem (siamo in una istanza dello schema generale con $a = b = 2$ e $f(n) = c$). Si osserva che $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. Quindi $f(n) = c = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ qualunque sia $0 < \epsilon < 1$. Quindi siamo nel primo caso del Master Theorem e si conclude che $T(n) = \Theta(n^{\log_b a}) = \Theta(n)$.

Domanda B (6 punti) Realizzare una funzione **SearchUnique(T,k)** che dato un albero binario di ricerca T (che si assume non vuoto), verifica se la chiave k è presente in un unico nodo, e, in caso affermativo restituisce il nodo, altrimenti (ovvero se la chiave non è presente oppure è presente in più nodi) restituisce **nil**. Si possono usare le funzioni standard degli alberi binari di ricerca. Valutarne la complessità.

Soluzione: L'idea è quella di cercare la chiave k con l'usuale ricerca dei BST. Se la chiave è presente, detto x il nodo trovato, per verificare se sia unica, si controlla che sottoalbero destro e sinistro non contengano ulteriori occorrenze della chiave. Questo si può fare con due ulteriori ricerche. La usuale funzione dei BST che cerca una chiave k nel sottoalbero radicato in x si indica con **Search(x,k)**.

```

SearchUnique(T,k)
  x = Search(T.root,k)

  if (x == nil) or (Search(x.left, k) <> nil) or (Search(x.right, k) <> nil)
    return x
  else
    return nil

```

L'algoritmo è corretto. L'osservazione fondamentale è che dato un nodo x di un BST, se la chiave k compare nel sottoalbero destro e in quello sinistro allora è anche la chiave della radice (infatti, per le proprietà dei BST, si avrebbe $k \leq x.key \leq k$). Ne segue che tutti i nodi con chiave k in T saranno nel sottoalbero del nodo x , ritornato dalla prima chiamata della funzione **Search**. Questo immediatamente implica la correttezza dell'algoritmo.

La complessità è quella di tre ricerche ciascuna con costo $O(h)$, dove h è l'altezza dell'albero, e quindi $O(h)$.

In alternativa, si può osservare che, se la chiave k dovesse comparire nel sottoalbero sinistro, sarebbe il massimo di questo sottoalbero e, dualmente, se comparisse nel secondo sottoalbero, sarebbe il minimo. Questo porta alla soluzione che segue

```

SearchUnique(T,k)
  x = Search(T.root,k)

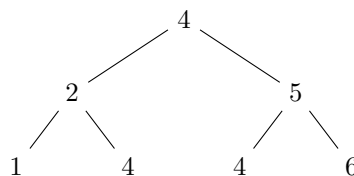
  if (x <> nil)
    y = Max(x.left)
    z = Min(x.right)

    if ((y <> nil) and (y.key == k)) or (z <> nil) and (z.key == k))
      return nil
    else
      return x
  else
    return nil

```

La complessità asintotica non cambia, ma questa funzione è più efficiente della precedente dato che **Min** e **Max**, diversamente a **Search** non eseguono confronti sulle chiavi.

Per concludere, è da osservare che l'assunzione che le chiavi duplicate siano solo nel sottoalbero destro (o in quello sinistro) o che siano adiacenti non è legittima. Ad esempio, il seguente è un BST valido:



Esercizi

Esercizio 1 (9 punti) Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno un campo aggiuntivo $x.min$, che rappresenta il minimo delle chiavi nel sottoalbero radicato in x . Realizzare la procedura `Insert(T,z)` che inserisce un nodo z nell'albero e la rotazione a sinistra `Left(T,x)` (assumendo che x e $x.right$ non siano `nil`). Valutarne la complessità.

Soluzione: La procedura di inserimento scende dalla radice fino ad una foglia dove il nodo viene inserito z . Dato che z sarà nel sottoalbero di ogni nodo attraversato durante la discesa, per ciascuno di questi nodi e soltanto per essi il campo *min* deve essere possibilmente aggiornato. Una piccola ottimizzazione: la verifica che $z.key < x.min$ ha senso farla solo se $z.key < x.key$ dato che $x.min \leq x.key$.

```
Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x

    if (z.key < x.key)
      if z.key < x.min      /* il sottoalbero radicato in z contiene z */
        x.min = z.key      /* quindi si aggiorna x.min                */

      x = x.left
    else
      x = x.right

  z.p = y
  if (y <> nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  z.min = z.key    /* il sottoalbero radicato in z contiene solo z */
```

La complessità rimane la stessa di *Insert* negli alberi binari di ricerca “standard”, ovvero $O(h)$, dove h è l'altezza dell'albero.

Anche la rotazione a sinistra si ottiene inserendo, in coda al codice standard, l'aggiornamento del campo *min*. Si osservi che gli unici nodi per i quali si modificano i nodi contenuti nel corrispondente sottoalbero sono x e $y = x.right$. I nodi che prima erano nel sottoalbero radicato in x poi sono in quello radicato in y , quindi i valori dei campi *min* e *max* per y dopo la rotazione sono quelli che prima erano in x . Per calcolare il nuovo valore del campo *min* per x si usa la conoscenza di tali valori per i figli destro e sinistro.


```

Left(T,z)
  y = x.right
  x.right = y.left
  x.right.p = x
  Transplant(T,x,y)
  y.left = x
  x.p = y

  y.min = x.min

  x.min = min(x.key, x.left.min, x.right.min)

```

La complessità resta costante $O(1)$.

Esercizio 2 (10 punti) Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza ℓ_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione. Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{j_i}(\sigma)$ del j_i -esimo programma è dato quindi dalle somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$. L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_{j=1}^n C_j(\sigma)$.

- Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
- Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Soluzione:

- Ordina i programmi per lunghezza crescente. Complessità: $O(n \log n)$.
- La scelta greedy consiste nello scegliere, come prossimo programma da eseguire, quello di lunghezza minima. Sia σ^* una soluzione ottima. Se il programma di lunghezza minima è il primo in σ^* , abbiamo finito. Consideriamo quindi il caso in cui il programma di lunghezza minima sia in posizione $k > 1$ in σ^* . Costruiamo una nuova soluzione σ' scambiando, in σ^* , il k -esimo programma con il primo. Possiamo osservare che:
 - l'insieme dei primi k programmi $j_1, j_2, \dots, j_k$ è lo stesso in σ^* e σ' , quindi il k -esimo programma ha lo stesso tempo di completamento in σ^* e σ' ; lo stesso vale per tutti i programmi successivi al k -esimo, visto che lo scambio non influisce su di loro;
 - per quanto riguarda tutti gli altri programmi, cioè quelli fino alla posizione $k-1$, questi hanno un tempo di completamento inferiore o uguale in σ' , perché lo scambio può solo avere ridotto la lunghezza del primo programma.

Quindi

$$\sum_{j=1}^n C_j(\sigma') \leq \sum_{j=1}^n C_j(\sigma^*);$$

siccome σ^* è una soluzione ottima, allora deve valere che

$$\sum_{j=1}^n C_j(\sigma') = \sum_{j=1}^n C_j(\sigma^*),$$

cioè anche σ' è una soluzione ottima.

Algoritmi e Strutture Dati

9 febbraio 2022

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegna la scansione dei fogli di bella copia, e come ultima pagina un documento di identità.

Domande

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
val(n)
  if n <= 2
    return 1
  else
    return val(n-1) + val(n-2) + val(n-2)
```

Determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Omega(2^n)$. La complessità è anche $O(2^n)$? Motivare le risposte.

Soluzione: Nel caso ricorsivo, si effettuano tre chiamate ricorsive, una con argomento $n - 1$ e due con argomento $n - 2$, e inoltre due somme (operazioni di tempo costante) si ottiene quindi

$$T(n) = T(n - 1) + 2T(n - 2) + c$$

Dimostriamo con il metodo di sostituzione che $T(n) = \Omega(2^n)$, ovvero dimostriamo che esistono $d > 0$ e n_0 tali che $T(n) \geq d \cdot 2^n$, per $n \geq n_0$. Si procede per induzione su n :

$$\begin{aligned} T(n) &= T(n - 1) + 2T(n - 2) + c && \text{[dalla definizione della ricorrenza]} \\ &\geq d2^{n-1} + 2d2^{n-2} + c && \text{[ipotesi induttiva]} \\ &\geq d2^{n-1} + 2d2^{n-2} && \text{[poiché } c > 0\text{]} \\ &= d(2^{n-1} + 2 \cdot 2^{n-2}) \\ &= d2^n \end{aligned}$$

$2^{n-1} + 2^{n-2+1}$
 $= 2^{n-1} + 2^{n-1}$
 $= 2 \cdot 2^{n-1} =$
 $2^{n-1+1} = 2^n \rightarrow d2^n$

qualunque siano d e n_0 .

Vale anche $T(n) = O(2^n)$, ovvero esistono $d > 0$ e n_0 tali che $T(n) \leq d2^n$ per un'opportuna costante $d > 0$ e $n \geq n_0$. Ai fini della prova induttiva è opportuno dimostrare che $T(n) \leq d(2^n - 1)$. Infatti:

$$\begin{aligned} T(n) &= T(n - 1) + 2T(n - 2) + c && \text{[dalla definizione della ricorrenza]} \\ &\leq d(2^{n-1} - 1) + 2d(2^{n-2} - 1) + c && \text{[ipotesi induttiva]} \\ &= d(2^{n-1} + 2 \cdot 2^{n-2}) - 3d + c \\ &= d2^n - 3d + c \\ &\leq d2^n \end{aligned}$$

dove l'ultima disuguaglianza vale se $-3d + c \leq 0$, e quindi per $d \geq c/3$.

Domanda B (6 punti) Si consideri un insieme di 6 attività $a_i, 1 \leq i \leq 6$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (2, 3, 5, 2, 1, 9)$$

$$\mathbf{f} = (4, 7, 8, 9, 10, 11).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: $\{a_1, a_3, a_6\}$

Esercizi

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *triangolare* se esiste $q \in [1, n]$ tale che $A[1..q]$ è ordinato in modo crescente e $A[q..n]$ è ordinato in modo decrescente. Realizzare una funzione $\text{maxTr}(A)$ che dato un array triangolare $A[1..n]$, senza elementi ripetuti, ne trova il massimo. Valutarne la complessità.

Soluzione: Si realizza una procedura divide et impera che opera sul sotto-array $A[p, r]$, che inizialmente sarà l'intero array. Se $p = r$, ovvero il sotto-array ha dimensione 1, semplicemente si ritorna l'unico elemento $A[p]$. Altrimenti, si considera il punto intermedio $q = \lfloor p + r \rfloor$. Se $A[q] < A[q + 1]$ significa che la parte $A[p..q]$ è crescente e la parte $A[q+1..r]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima e può essere cercato ricorsivamente. Dualmente se $A[q] > A[q + 1]$ significa che la parte $A[q+1..r]$ è decrescente e la parte $A[p..q]$ è ancora triangolare, per cui il massimo si troverà in quest'ultima.

```
maxTr(A,p,r)                                12348675
    if p=r
        return A[p]
    else
        q = (p+r)/2
        if A[q]<A[q+1]
            return maxTr(A,q+1,r)
        else
            return maxTr(A,p,q)

maxTr(A)
    return maxTr(A,1,A.length)
```

La complessità è espressa dalla ricorrenza

$$T(n) = T(n/2) + c$$

Si può risolvere utilizzando il master theorem, confrontando $n^{\log_b a} = n^{\log_2^2} = n$ con $f(n) = c$. Dato che $f(n) = O(n^{\log_b a - \epsilon}) = O(n^{1-\epsilon})$ per $0 < \epsilon < 1$ si conclude che siamo nel primo caso del teorema che ci dà la soluzione $T(n) = \Theta(\log n)$.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j - 1) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

- (a) Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```
COMPUTE(a,b)
n <- length(a)
m = +infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MIN(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] * C[i,j+1]
    m <- MIN(m,C[i,j])
return m
```

- (b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n - 1 - i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Algoritmi e Strutture Dati

17 giugno 2022

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 4T(n/3) + 2n^2 + 1$$

Soluzione: Rispetto allo schema generale si ha $a = 4$, $b = 3$, $f(n) = 2n^2 + 1$. Si osserva che $\log_b a = \log_3 4 < 2$ quindi $f(n) = \Omega(n^{\log_b a + \epsilon})$ (per $0 < \epsilon \leq 2 - \log_3 4$). In aggiunta vale la condizione di regolarità, ovvero $af(\frac{n}{b}) \leq cf(n)$ per qualche $c < 1$. Infatti $af(\frac{n}{b}) = 4f(\frac{n}{3}) = 4(\frac{2n^2}{9} + 1) \leq 2cn^2 \leq c(2n^2 + 1)$, asintoticamente, quando $4/9 < c < 1$. Più precisamente, per avere

$$4(\frac{2n^2}{9} + 1) \leq 2cn^2$$

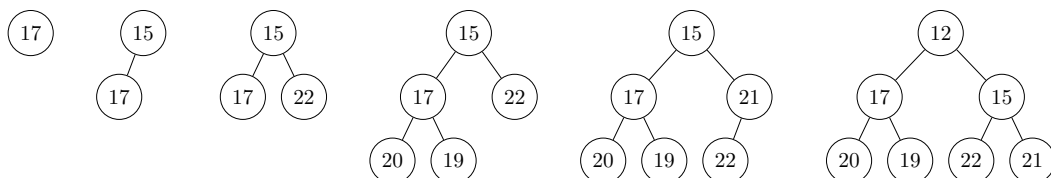
Occorre che $c \geq \frac{4}{9} + \frac{2}{n^2}$, che, per $n \geq 2$, si ottiene quando $c \geq \frac{4}{9} + \frac{2}{4} = \frac{15}{18} < 1$.

Domanda B (7 punti) Dare la definizione di min-heap. Data la sequenza di elementi 17, 15, 22, 20, 19, 21, 12, si specifichi il min-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 17. Si descriva sinteticamente come si procede per arrivare al risultato.

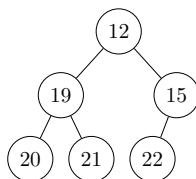
Soluzione: Un min-heap è un albero binario ordinato quasi-completo con la proprietà che per ogni nodo x , se x non è radice, il genitore di x ha chiave minore o uguale a quella di x , o equivalentemente ogni nodo x ha chiave minore o uguale a quella dei suoi successori.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MinHeapifyUp** per ripristinare la proprietà di min-heap.

Gli inserimenti nell'ordine producono gli heap indicati in figura



Infine, la rimozione di un nodo con chiave x si realizza sostituendolo con l'ultima foglia y e richiama **MinHeapifyUp**, se $y < x$ oppure **MinHeapify**, se $y > x$. Nello specifico per eliminare 17 lo si rimpiazza con l'ultima foglia 21 e si richiama **MinHeapify**, ottenendo il min-heap



che in forma di array è $[12, 19, 15, 20, 21, 22]$.

Esercizi

Esercizio 1 (10 punti) Diciamo che un array senza ripetizioni $A[1, n]$ è *semi-ordinato* se esiste un indice k , con $1 \leq k < n$, tale che $A[k+1..n]$ sia ordinato, ovvero i sottoarray $A[k+1..n]$ e $A[1..k]$ sono ordinati e $A[n] < A[1]$. In questo caso l'indice k viene detto il centro dell'array. Ad esempio l'array che segue è semi-ordinato con centro $k = 4$.

1	2	3	4	5	6	7
4	9	12	18	-1	1	2

Scrivere una funzione **centre(A)** che dato un array A semi-ordinato ne restituisce il centro. Giustificare la correttezza dell'algoritmo e valutarne la complessità.

Soluzione: L'idea è quella di procedere con un algoritmo divide et impera. A tal fine osservazione fondamentale è la seguente: dato un sottoarray $A[p..r]$ semi-ordinato, se lo divido in due sottoarray $A[p..q]$ e $A[q..r]$, allora uno solo dei due è semi-ordinato (quello che contiene il centro), mentre l'altro è ordinato. Quando la dimensione del sottoarray $A[p..r]$ diventa 2, il centro è p .

Lo pseudo-codice è quindi il seguente:

```

centre(A)
    return centre-rec(A, 1, A.length)

centre-rec(A, p, r)
    if r == p+1
        return p
    else
        q = (p+r)//2
        if (A[q] < A[p])
            return centre-rec(A, p, q)
        else
            return centre-rec(A, q, r)

```

La prova di correttezza procede per induzione sul numero di elementi n dell'array $A[p..r]$

- Se $n = 2$, ovvero $r = p + 1$, allora $A[p + 1] < A[p]$ e chiaramente p è il centro.
- Se invece $n > 2$, l'array viene diviso in due parti, ovvero si definisce $q = \lfloor (p+r)/2 \rfloor$ e si considerano gli array $A[p..q]$ e $A[q..r]$. Detto k il centro, ci sono due possibilità.
 - ($q \leq k$) Questo accade se e solo se $A[p] > A[q]$, e correttamente in questo caso si ricorre sul sottoarray $A[q..r]$
 - ($q > k$) Questo accade se e solo se $A[p] < A[q]$, e correttamente in questo caso si ricorre sul sottoarray $A[p..q]$

Per quanto riguarda la complessità, la ricorrenza è del tipo $T(n) = T(n/2) + c$, dato che ad ogni chiamata si dimezza la dimensione dell'array e il costo della parte non ricorsiva è costante. Per il Master Theorem (con $a = 1$, $b = 2$, $f(n) = c = \Theta(n^{\log_b a}) = \Theta(n^0) = \Theta(1)$) si conclude $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$.

Esercizio 2 (9 punti) Sia n un intero positivo. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 2 & \text{se } i = j, \\ 3 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) + M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

- (a) Si scriva una coppia di algoritmi INIT_M(n) e REC_M(i, j) per il calcolo memoizzato di $M(1, n)$.
- (b) Si calcoli il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Soluzione:

(a) INIT_M(n)

```

if n=1 then return 2
if n=2 then return 3
for i=1 to n-1 do
    M[i,i] = 2
    M[i,i+1] = 3
M[n,n] = 2
for i=1 to n-2 do
    for j=i+2 to n do
        M[i,j] = 0
return REC_M(1,n)

```

REC_M(i, j)

```

if M[i,j] = 0 then
    M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) + REC_M(i,j-1)
return M[i,j]

```

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 = \sum_{i=1}^{n-2} (n - i - 1) = \sum_{k=1}^{n-2} k = (n-2)(n-1)/2$$

Algoritmi e Strutture Dati

4 Luglio 2022

Cognome Nome Matricola

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea sottostante e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{1}{4} T(n-1) + 3n^2$$

ha soluzione in $\Theta(n^2)$.

Soluzione: Per provare che $T(n) = O(n^2)$ dobbiamo dimostrare che $T(n) \leq cn^2$, per un'opportuna costante $c > 0$. Procediamo per induzione:

$$\begin{aligned} T(n) &= 1/4 T(n-1) + 3n^2 \\ &\leq 1/4 c(n-1)^2 + 3n^2 \\ &\leq 1/4 cn^2 + 3n^2 \\ &\leq cn^2 \end{aligned}$$

dove, per la validità dell'ultima disuguaglianza $1/4 cn^2 + 3n^2 \leq cn^2$, occorre che

$$3/4 cn^2 \geq 3n^2$$

ovvero, $c \geq 4$, con n qualunque.

Per provare $T(n) = \Omega(n^2)$ dobbiamo dimostrare che $T(n) \geq dn^2$, per un'opportuna costante $d > 0$. La prova per induzione non usa neppure l'ipotesi induttiva. Infatti

$$T(n) = 1/4 T(n-1) + 3n^2 \geq dn^2$$

purché $d \leq 3$.

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *oro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	<i>o</i>	<i>r</i>	<i>o</i>
	0	0	0
<i>a</i>	0	0	0
<i>r</i>	0	0	1
<i>m</i>	0	0	1
<i>o</i>	0	1	2

La lunghezza della longest common subsequence tra *armo* e *oro* è quindi 2.

Esercizi

Esercizio 1 (9 punti) Realizzare una funzione `strongBST(T)` che dato un albero binario `T` con chiavi numeriche non negative, verifica se, per ogni nodo, la chiave è maggiore uguale del doppio di ogni chiave nel sottoalbero sinistro e minore o uguale della metà di ogni chiave nel sottoalbero destro, e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo `x` ha i campi `x.left`, `x.right` e `x.key`). Valutarne la complessità.

Soluzione:

```
strongBST(T)
    s, m, M = strongBST(T.root)
    return v

# strongBSTRec(x):
# verifica se il sottoalbero radicato in x e' uno strongBST e ritorna tre valori:
# - un booleano (che indica l'esito della verifica)
# - il massimo delle chiavi nel sottoalbero sx
# - il minimo delle chiavi nel sottoalbero dx

strongBSTRec(x)

if x = nil
    return true, 0, +infty
else
    # ispeziono i sottoalberi sinistro e destro
    sl, ml, Ml = strongBSTRec(x.left)
    sr, mr, Mr = strongBSTRec(x.right)

    # se la chiave del nodo in esame e' maggiore o uguale del doppio
    # del massimo delle chiavi nel sottoalbero sinistro (quindi di
    # tutte le chiavi nel sottoalbero sinistro) e minore o uguale della
    # meta' del minimo delle chiavi nel sottoalbero sinistro (quindi di
    # tutte le chiavi nel sottoalbero destro) delle chiavi dei
    # discendenti, ritorna true
    s = (x.key >= 2*Ml) and (x.key <= 1/2 mr)

    # calcola il minimo e il massimo delle chiavi nel sottoalbero radicato in x
    m = min { ml, mr, x.key }
    M = max { Ml, Mr, x.key }

    return s, m, M
```

Si tratta di una visita e quindi la complessità è $\Theta(n)$.

Esercizio 2 (10 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k

sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un (2,1)-boxing che minimizza il numero di sottoinsiemi della partizione.

1. Scrivere il codice di un algoritmo greedy che restituisce un (2,1)-boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

Soluzione:

1. L'idea è provare ad accoppiare il numero più piccolo (a_1) con quello più grande (a_n). Se la loro somma è al massimo 1, allora $S_1 = \{a_1, a_n\}$, altrimenti $S_1 = \{a_n\}$. Poi si procede analogamente sul sottoproblema $S \setminus S_1$.

```
(2,1)-BOXING(S)
n <- |S|
P <- empty_set
first <- 1
last <- n
while (first <= last)
  if (first < last) and a_first + a_last <= 1 then
    P <- P U {{a_first, a_last}}
    first <- first + 1
  else
    P <- P U {{a_last}}
    last <- last - 1
return P
```

Questo algoritmo scansiona ogni elemento una sola volta, quindi la sua complessità è lineare.

2. La scelta greedy è $\{a_1, a_n\}$ se $n > 1$ e $a_1 + a_n \leq 1$, altrimenti $\{a_n\}$. Ora dimostriamo che esiste sempre una soluzione ottima che contiene la scelta greedy. I casi $n = 1$ e $a_1 + a_n > 1$ sono banali, visto che in questi casi ogni soluzione ammissibile deve contenere il sottoinsieme $\{a_n\}$. Quindi assumiamo che la scelta greedy sia $\{a_1, a_n\}$. Consideriamo una qualsiasi soluzione ottima dove a_1 e a_n non sono accoppiati nello stesso sottoinsieme. Quindi, esistono due sottoinsiemi S_1 e S_2 , con $a_1 \in S_1$ e $a_n \in S_2$. Sostituiamo questi due sottoinsiemi con $S'_1 = \{a_1, a_n\}$ (cioè, la scelta greedy) e $S'_2 = S_1 \cup S_2 \setminus \{a_1, a_n\}$. $|S'_2| \leq 2$ e, se $|S'_2| = 2$, allora $S'_2 = \{a_s, a_t\}$ con $a_s \in S_1$ e $a_t \in S_2$. Siccome a_t era precedentemente accoppiato con a_n , a maggior ragione può essere accoppiato con $a_s < a_n$, quindi la nuova soluzione così creata è ammissibile e ancora ottima.

Algoritmi e Strutture Dati

12 Settembre 2022

Cognome Nome Matricola

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea sottostante e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Definire formalmente la classe $\Omega(f(n))$. Dimostrare che la seguente ricorrenza ha soluzione $T(n) = \Omega(n)$

$$T(n) = \frac{1}{3}T(n-1) + 2n + 1$$

Soluzione: L'insieme $\Omega(f(n))$ è definito come:

$$\Omega(f(n)) = \{g(n) : \exists c > 0. \exists n_0. \forall n \geq n_0. 0 \leq cf(n) \leq g(n)\}.$$

Si deve provare che asintoticamente, per un'opportuna costante $c > 0$

$$T(n) \geq cn$$

Si procede per induzione:

$$\begin{aligned} T(n) &= \frac{1}{3}T(n-1) + 2n + 1 && [\text{per definizione della ricorrenza}] \\ &\geq \frac{1}{3}cn + 2n + 1 && [\text{per ipotesi induttiva } T(n-1) \geq c(n-1)] \\ &\geq 2n \\ &\geq cn \end{aligned}$$

dove l'ultima disuguaglianza vale quando $c \leq 2$. Risulta dunque dimostrata la tesi.

Domanda B (6 punti) Si consideri una tabella hash di dimensione $m = 8$, gestita mediante chaining (liste di trabocco) con funzione di hash $h(k) = k \bmod m$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 28, 19, 10, 35, 26.

Soluzione: La tabella hash T contiene, in corrispondenza di ciascuna entry $T[i]$ la lista degli elementi x tali che $h(x.key) = i$. L'inserimento in testa alla lista garantisce complessità dell'inserimento $O(1)$.

Si ottiene

0	☐	
1	☐	
2	☐	$\rightarrow 26 \rightarrow 10$
3	☐	$\rightarrow 35 \rightarrow 19$
4	☐	$\rightarrow 28$
5	☐	
6	☐	
7	☐	

Esercizio 1 (10 punti) Realizzare una funzione `Diff(A,k)` che, dato un array `A[1,n]` ordinato in senso decrescente, verifica se esiste una coppia di indici `i, j` tali che `A[i] - A[j] = k`. Restituisce la coppia di indici se esiste e (0,0) altrimenti. La funzione non deve alterare l'input e deve operare in spazio costante. Scrivere lo pseudocodice, provarne la correttezza e valutarne la complessità.

Il codice può essere:

È facile vedere che si mantiene l'invariante $\forall (i', j') \in [1, n]. (i' < i) \vee (j' < j) \Rightarrow A[i] - A[j] \neq k$, ovvero una coppia (i', j') tale che $A[i] - A[j] = k$ può esistere solo tra le coppie ancora esplorabili ($i' \geq i$ e $j' > j$), ovvero, graficamente nella parte non grigia:

[illegible]

Ad ogni iterazione, se entro nel ciclo, ci sono due possibilità:

- $$A[i'] - A[j] \leq A[i] - A[j] < k.$$

$$A[i] - A[j'] \geq A[i] - A[j] > k.$$

Dunque, anche in questo caso, non escludo coppie utili, l'invariante continua a valere.

Quando esco dal ciclo, se $A[i] - A[j] = k$, ho concluso con successo. Altrimenti deve essere $i > n$ o $j > n$, che unitamente all'invariante, mi permettono di concludere che per ogni $i, j \in [1, n]$, $A[i] - A[j] \neq k$, come desiderato.

Da questo la correttezza segue immediatamente. La complessità è lineare. Il numero di iterazioni è pari al più a $2n - 1$, dato che i e j partono da 1, sono limitate da n ed ogni iterazione aumenta una delle due. Dato che ciascuna iterazione ha costo costante, ottengo $T(n) = O(2n - 1) = O(n)$.

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $c(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$c(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ c(i-1, j) \cdot c(i, j+1) \cdot c(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up `COMPUTE_C(A)` che, data in input la stringa A restituisca in uscita il valore $c(n, 1)$.
2. Si valuti il numero esatto $T_{CC}(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
COMPUTE_C(A)
n = length(A)
for i=1 to n do
    c[1,i] = a_i
    c[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        c[i,j] = c[i-1,j] * c[i,j+1] * c[i-1,j+1]
return c[n,1]
```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $c[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due operazioni tra interi, e quindi

$$\begin{aligned} T_{CC}(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

Algoritmi e Strutture Dati

30 gennaio 2023

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{3}{4}T(n/3) + T(2n/3) + 2n$$

ha soluzione in $\Theta(n)$.

Soluzione: Per provare che $T(n) = O(n)$ dobbiamo dimostrare che $T(n) \leq cn$, per un'opportuna costante $c > 0$. Procediamo per induzione:

$$\begin{aligned}T(n) &= 3/4 T(n/3) + T(2n/3) + 2n \\&\leq 3/4 c(n/3) + c(2n/3) + 2n \quad [\text{per ipotesi induttiva}] \\&= 11/12 cn + 2n \\&\leq cn\end{aligned}$$

dove, per la validità dell'ultima disuguaglianza $11/12 cn + 2n \leq cn$, occorre che

$$1/12 cn \geq 2n$$

ovvero, $c \geq 24$, con n qualunque.

Per provare $T(n) = \Omega(n)$ dobbiamo dimostrare che $T(n) \geq dn$, per un'opportuna costante $d > 0$. La dimostrazione è più semplice della precedente

$$\begin{aligned}T(n) &= 3/4 T(n/3) + T(2n/3) + 2n \\&\geq 2n \geq dn\end{aligned}$$

Quindi è sufficiente scegliere $0 < d \leq 2$, e la relazione vale per n qualunque.

Domanda B (6 punti) Calcolare la lunghezza della longest common subsequence (LCS) tra le stringhe *store* e *shoes*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	s	h	o	e	s
	0	0	0	0	0
s	0	1	1	1	1
t	0	1	1	1	1
o	0	1	1	2	2
r	0	1	1	2	2
e	0	1	1	2	3

La lunghezza della longest common subsequence tra *store* e *shoes* è quindi 3.

Esercizi

Esercizio 1 (10 punti) Un array di interi $A[1..n]$ si dice *3-ordinato* se per ogni coppia di indici $i, j \in [1, n]$ con $i \leq j$ vale $A[i] \% 3 \leq A[j] \% 3$, dove $k \% 3$ indica il resto della divisione intera di k per 3. Realizzare una funzione `3Order(A)` che dato un array $A[1..n]$, lo rende 3-ordinato. Valutarne la complessità in tempo e in spazio, e indicare se l'algoritmo è stabile. Anche qualora si usasse un algoritmo di ordinamento noto, lo pseudo-codice va comunque scritto esplicitamente.

Soluzione: Un'osservazione semplice ma fondamentale è che di fatto si vuole ordinare l'array ottenuto sostituendo ogni valore $A[i]$ con $A[i] \% 3$.

Si può dunque ricorrere un algoritmo di ordinamento classico, sostituendo, nel confronto, i valori $A[i]$ con $A[i] \% 3$, ovvero con il loro resto modulo 3.

Ad esempio, se si usa il *Mergesort* si avrà complessità di tempo $O(n \log n)$ e spazio $O(n)$, e l'algoritmo è stabile:

```
3Order (A, p, r)
    if p < r
        q = (p+r)/2
        3Order (A, p, q)
        3Order (A, q+1, r)
        Merge (A, p, q, r)

Merge (A, p, q, r)
    n1 = q-p+1
    n2 = r-q
    for i = 1 to n1
        L[i] = A[i]
    for j = 1 to n2
        R[j] = A[p+j]

    L[n1+1]=R[n2+1] = infinity

    i=p
    j=q+1
    for k = p to r
        if (A[i]%3 <= B[j]%3): // si assume infinity%3 = infinity
            A[k] = L[i]
            i++
        else
            A[k]=B[j]
            j++
    }
```

Tuttavia si può osservare che $A[i]\%3$ ha valori in $\{0, 1, 2\}$ così che si può facilmente utilizzare un counting sort, con complessità di tempo $O(n)$ e spazio $O(n)$, stabile.

```
3Order (A, B, n)
  allocate C[0..2]

  for i=0 to 2
    C[i]=0

  for j=1 to n
    C[A[j]]++

  for i=1 to 2
    C[i]=C[i-1]+C[i]

  for j=n downto 1
    B[C[A[j]]] = A[j]
    C[A[j]]--
```

La soluzione migliore è probabilmente operare una tripartizione, come visto per il QuickSort, sulla base del valore di $A[i]\%3$. Concretamente si scorre l'array con un indice k mantenendo tre partizioni, corrispondenti ai valori 0, 1 e 2 per $A[i]\%3$: valore 0 in $A[1..p-1]$, valore 1 in $A[p..k-1]$, valore 2 in $A[j..n]$. La complessità di tempo è ancora $O(n)$ ma l'algoritmo è in place, quindi spazio $O(1)$. L'algoritmo non è stabile.

```
3Order (A,n)
  i = 0
  k = 1
  j = n+1
  while k < j

    if A[k]%3 == 0
      i++
      A[k] <-> A[i]
      k++

    else if A[k]%3 == 1
      k++

    else // A[k]%3 == 2
      j--
      A[k] <-> A[j]
```

Esercizio 2 (9 punti) Supponiamo di avere un numero illimitato di monete di ciascuno dei seguenti valori: 50, 20, 1. Dato un numero intero positivo n , l'obiettivo è selezionare il più piccolo numero di monete tale che il loro valore totale sia n . Consideriamo l'algoritmo greedy che consiste nel selezionare ripetutamente la moneta di valore più grande possibile.

- Fornire un valore di n per cui l'algoritmo greedy *non* restituisce una soluzione ottima.
- Supponiamo ora che i valori delle monete siano 10, 5, 1. In questo caso l'algoritmo greedy restituisce sempre una soluzione ottima: dimostrare che ogni insieme ottimo M^* di monete di valore totale n contiene la scelta greedy.

Soluzione:

- (a) Per esempio $n = 60$, perché la soluzione ottima è 3 monete da 20, mentre l'algoritmo greedy restituisce 11 monete (una da 50 e 10 da 1).
- (b) Sia M^* una soluzione ottima. Sia x il valore maggiore tra 10, 5, e 1 che sia non superiore a n . Se M^* contiene una moneta di valore x , la proprietà è dimostrata. Altrimenti, sia $M \subseteq M^*$ un insieme di (2 o più) monete di valore totale x (si osservi che tale insieme esiste sempre quando i valori delle monete sono 10, 5, 1); consideriamo $M' = M^* \setminus M \cup X$, dove X è l'insieme contenente una moneta di valore x . M' è un insieme di monete di valore totale n e di cardinalità inferiore a quella di M^* : assurdo, quindi questo secondo caso non può verificarsi, e quindi M^* contiene necessariamente una moneta di valore x .

Algoritmi e Strutture Dati

21 febbraio 2023

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti) Dare la definizione formale delle classi $O(f(n))$ e $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che se $f(n) = O(n^2)$ e $g(n) = \Omega(n)$, con $g(n) > 0$ per ogni n , allora $f(n)/g(n) = O(n)$.

Soluzione: Le classi $O(f(n))$ e $\Omega(g(n))$ sono definite come:

$$\begin{aligned} O(f(n)) &= \{g(n) : \exists c > 0. \exists n_0. \forall n \geq n_0. 0 \leq g(n) \leq cf(n)\} \\ \Omega(f(n)) &= \{g(n) : \exists d > 0. \exists n_0. \forall n \geq n_0. 0 \leq d f(n) \leq g(n)\} \end{aligned}$$

Si assuma che $f(n) = O(n^2)$ e $g(n) = \Omega(n)$. Ovvero, esistono $c > 0, n_0$ tali che per ogni $n \geq n_0$

$$0 \leq f(n) \leq cn^2$$

e $d > 0, m_0$ tali che per ogni $n \geq m_0$

$$0 < dn \leq g(n)$$

Quindi, per $n \geq \max\{n_0, m_0\}$ si ha che

$$\begin{aligned} 0 &\leq f(n)/g(n) \\ &\leq cn^2/g(n) && [\text{poiché } f(n) \leq cn^2] \\ &\leq cn^2/(dn) && [\text{poiché } g(n) \geq dn] \\ &= (c/d)n. \end{aligned}$$

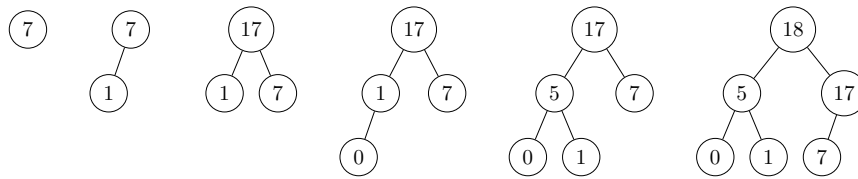
dato che $c/d > 0$ questo conclude la prova.

Domanda B (7 punti) Dare la definizione di max-heap. Data la sequenza di elementi 7, 1, 17, 0, 5, 18, si specifichi il max-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 0. Si descriva sinteticamente come si procede per arrivare al risultato.

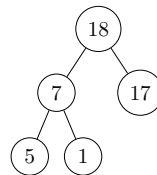
Soluzione: Un max-heap è un albero binario ordinato quasi-completo (tutti i livelli completi, a parte l'ultimo, nel quale le foglie devono essere "addossate" a sinistra) con la proprietà che per ogni nodo x , se x non è radice, gli antenati di x hanno chiave maggiore o uguale a quella di x , o equivalentemente ogni nodo x ha chiave maggiore o uguale a quella dei suoi discendenti.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MaxHeapifyUp** per ripristinare la proprietà di min-heap (scambiando il nodo con il suo genitore finché la chiave di questo è inferiore a quella del figlio).

Gli inserimenti nell'ordine producono gli heap indicati in figura



Infine, la rimozione di un nodo con chiave x si realizza sostituendolo con l'ultima foglia y e richiamando **MaxHeapifyUp**, se $y > x$ oppure **MaxHeapify**, se $y < x$. Nello specifico per eliminare 0 lo si rimpiazza con l'ultima foglia 7 e si richiama **Max**, ottenendo il min-heap



che in forma di array è $[12, 19, 15, 20, 21, 22]$.

Esercizi

Esercizio 1 (10 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , il grado di bilanciamento del sottoalbero radicato in x , definito come $h_x / \log_2(n_x + 1)$ dove h_x e n_x indicano rispettivamente l'altezza e il numero di nodi del sottoalbero radicato in x (si intende che un albero costituito da un solo nodo abbia altezza 1).

Indicare quali campi occorre aggiungere ai nodi. Fornire lo pseudo-codice per la funzione **bal(x)** che restituisce il grado di bilanciamento del sottoalbero radicato in x e la procedura di inserimento di un nodo **insert(T,z)**. Valutare la complessità delle funzioni definite.

Soluzione: L'idea è quella di arricchire la struttura facendo in modo che ogni nodo x , oltre ai campi usuali, abbia due campi aggiuntivi: $x.h$ che contiene l'altezza del sottoalbero radicato in x e $x.size$ che contiene il numero dei nodi in tale sottoalbero.

A questo punto è immediato realizzare la funzione **bal(x)** di tempo costante

```
bal(x)
  if x <> nil
    return x.h/log_2(x.size)
  else
    error
```

Invece per l'inserimento, si modifica la procedura standard, che discende l'albero dalla radice fino alla posizione in cui inserire il nuovo nodo, facendo in modo che tutti i nodi attraversati e che quindi conterranno nel sottoalbero il nuovo nodo, abbiano il campo **size** incrementato di 1. Inoltre, una eventuale modifica dell'altezza va propagata verso l'alto. Questo accade quando il parent del nodo inserito era una foglia e quindi la sua altezza, prima dell'inserimento, era 1.

```

Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x
    x.size++
    if (z.key < x.key)
      x = x.left
    else
      x = x.right

  // inizializza i campi del nodo z
  z.size = 1
  z.left = z.right = nil
  z.h = 1

  z.p = y
  if (y <> nil)      // se z non radice
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  // se y era una foglia, ovvero se l'altezza di y era 1, la sua
  // altezza e conseguentemente quella degli antenati va
  // aumentata di 1 risalendo i nodi attraversati nella discesa
  if y.h == 1
    while y <> nil
      y.h ++
      y = y.p

```

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Esercizio 2 (9 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j - 1) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $m = \max\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

- Fornire un algoritmo iterativo bottom-up per il calcolo di m .
- Valutare la complessità *esatta* dell'algoritmo, associando costo unitario alle moltiplicazioni tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

- (a) Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = -infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MAX(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MAX(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] * C[i,j+1]
    m <- MAX(m,C[i,j])
return m

```

- (b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} (n-1-i) = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$